

Tài liệu hướng dẫn Đồ án Big Data

NHẬP MÔN DỮ LIỆU LỚN (2024-2025)

Duong Thai Bao

INDUSTRIAL UNIVERSITY OF HO CHI MINH CITY | PHIÊN BẢN TÀI LIỆU: 251504-BIGDATA-IUH-4A

MỤC LỤC

LỜI MỞ ĐẦU.....	3
Phần 1: Xây dựng hệ thống mạng ảo giả lập trên Virtual Box.....	4
Các cấu hình mạng:	4
<i>NAT:</i>	<i>4</i>
<i>Internal Network Mode:.....</i>	<i>4</i>
<i>Bridged Network Mode</i>	<i>5</i>
<i>Host-only Networking</i>	<i>5</i>
Mô tả bài toán:	6
<i>Câu hỏi: Những máy bên ngoài có thể truy xuất vào bên trong hay không?</i>	<i>6</i>
<i>Chuẩn bị:</i>	<i>6</i>
<i>Thực hiện:</i>	<i>7</i>
Hướng dẫn cấu hình mạng Nat và Host-Only Network cho máy master01.....	8
Kiểm tra cụm 5 máy tính.....	10
Task:	10
Phần 2: Kiểm tra các gói phần mềm trong hệ thống với Docker Engine	11
Thực hành: Kiểm tra các gói cài đặt phần mềm.....	11
<i>Mô tả:</i>	<i>11</i>
<i>Các gói phần mềm:</i>	<i>11</i>
<i>Trong Docker Compose có config gì:</i>	<i>12</i>
<i>Test như thế nào?</i>	<i>12</i>
Phần 3: Khám phá hiện tượng Bottleneck trong hệ thống Microservices.....	14
Thực hành: Khám phá hiện tượng nghẽn cổ chai	14
<i>Làm sao để biết nghẽn cổ chai, nghẽn cổ chai là gì?</i>	<i>14</i>
<i>Bắt đầu thực hành:.....</i>	<i>14</i>
Phần 4: Xây dựng cụm máy tính dựa trên Docker Swarm.....	19
Thực hành: Khám phá Docker Swarm (Cụm máy docker)	19
<i>[Đọc thêm] SwarmKit là gì?</i>	<i>23</i>
<i>[Đọc thêm] Tạo Token (Token Generation)</i>	<i>24</i>
Phần 5: HIGH AVAILABILITY & AUTO SCALING & LOAD BALANCING IN DOCKER SWARM.....	25

Thực hành: Quản trị và vận hành cụm máy docker	25
Phần 6: Hướng dẫn cài đặt ELK Stack trên Docker Swarm.....	34
Phần 7: Giám sát Docker Swarm	39

LỜI MỞ ĐẦU

Đây là tài liệu được biên soạn lại dựa trên những tài liệu và hướng dẫn của thầy Huỳnh Nam, với mục đích hỗ trợ nhóm trong quá trình làm đồ án.

Tài liệu được sử dụng **nội bộ, phục vụ mục đích học tập**.

Trong mọi trường hợp phát sinh như sử dụng trong thương mại, hoặc các vấn đề phát sinh liên quan đến bản quyền, phát sinh khác, người sử dụng tài liệu trái phép sẽ tự chịu trách nhiệm với hành động của mình.

Khuyến nghị bạn nên tham khảo những nội dung gốc từ Thầy Huỳnh Nam [tại đây](#).

Trong quá trình sử dụng tài liệu, nếu có sai sót hoặc vấn đề xảy ra, rất mong nhận được sự góp ý từ bạn. Bạn có thể liên hệ với người soạn tài liệu này tại: bigdata-iuh-doc@dtbao.io.vn.

Trân trọng cảm ơn,

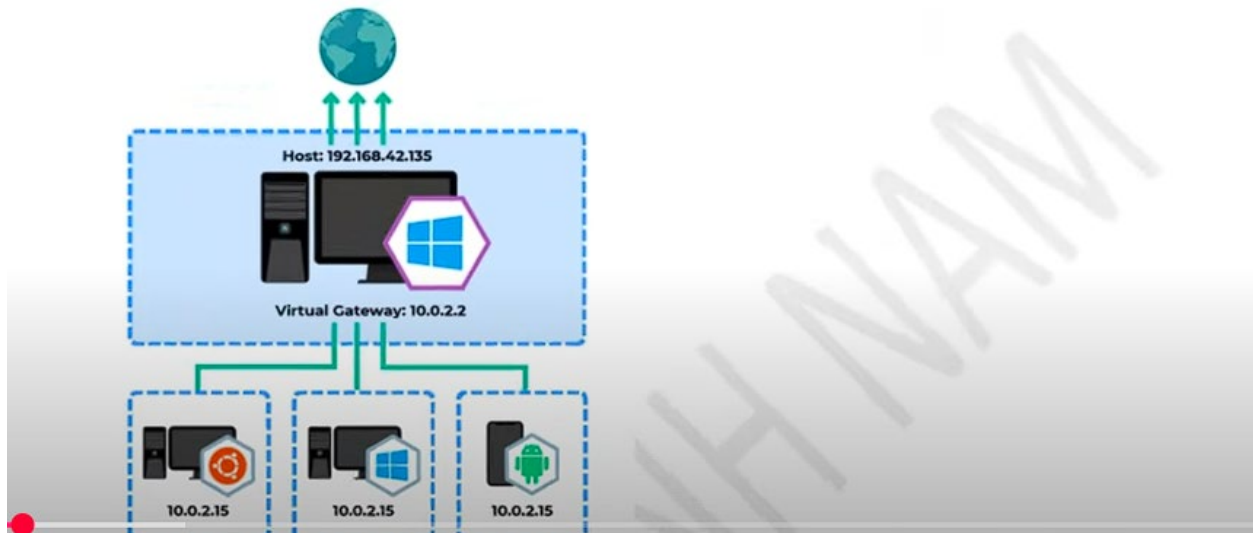
Dương Thái Bảo

Phần 1: Xây dựng hệ thống mạng ảo giả lập trên Virtual Box

Các cấu hình mạng:

NAT:

As previously mentioned, the default network mode is NAT. It allows VMs to access the internet (if their host is connected), but not to communicate with each other.



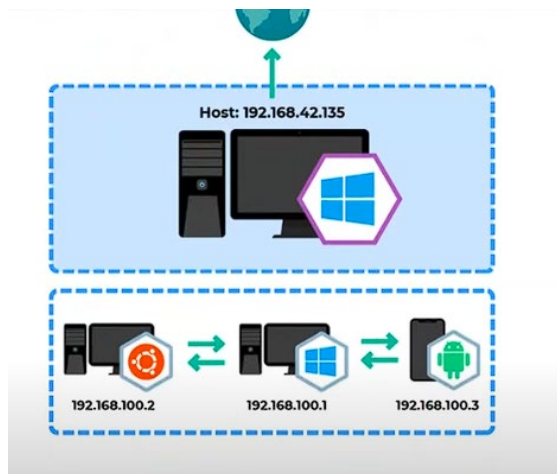
- 3 máy ảo sẽ có cùng IP, có thể kết nối ra ngoài internet
- Thông qua gateway cố định 10.0.2.2
- Từ máy windows không truy cập ngược được do IP giống nhau.
- Các máy đều giống IP nên không thể giao tiếp với nhau.

Internal Network Mode:

The simplest way to establish communication between the VMs is to put them all on an internal network using the internal network mode.

Internal does not have a DHCP server on it by default. So a VM adapter attached to Internal won't get an IP address, unless you set the adapter to static IP in the VM's OS or provide a DHCP server on the Internal network, either through another VM or by manually enabling a DHCP server through Vboxmanage.

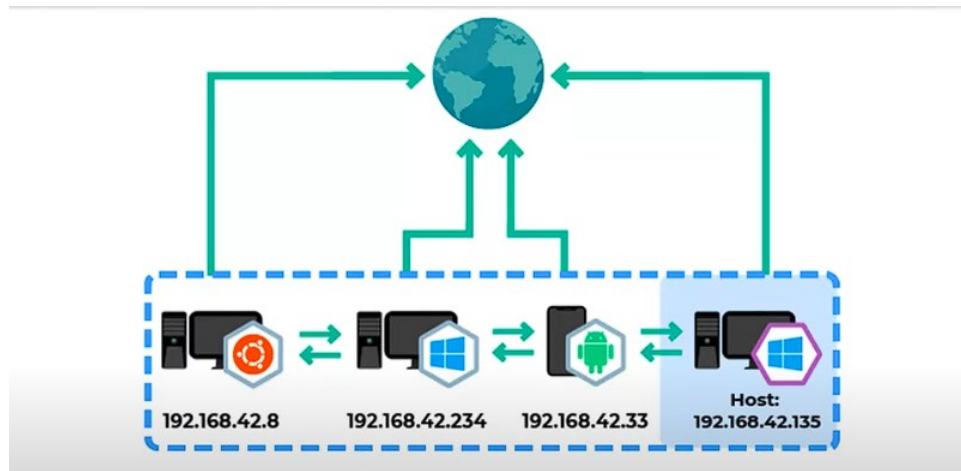
Các máy ảo chỉ có thể nói chuyện với nhau, có thể có một IP tĩnh



Giao tiếp qua ngoài Internet, hay từ ngoài vào máy ảo không được

Bridged Network Mode

Use bridge network mode if you want to allow your VMs to access the internet, communicate with each other, and be visible externally.



Mỗi một máy ảo sẽ được cấp 1 IP tương đương với host, trên hệ thống mạng chung của máy host đang dùng. DHCP sẽ cấp cho các máy ảo, giao tiếp được giữa các máy, ra Internet.

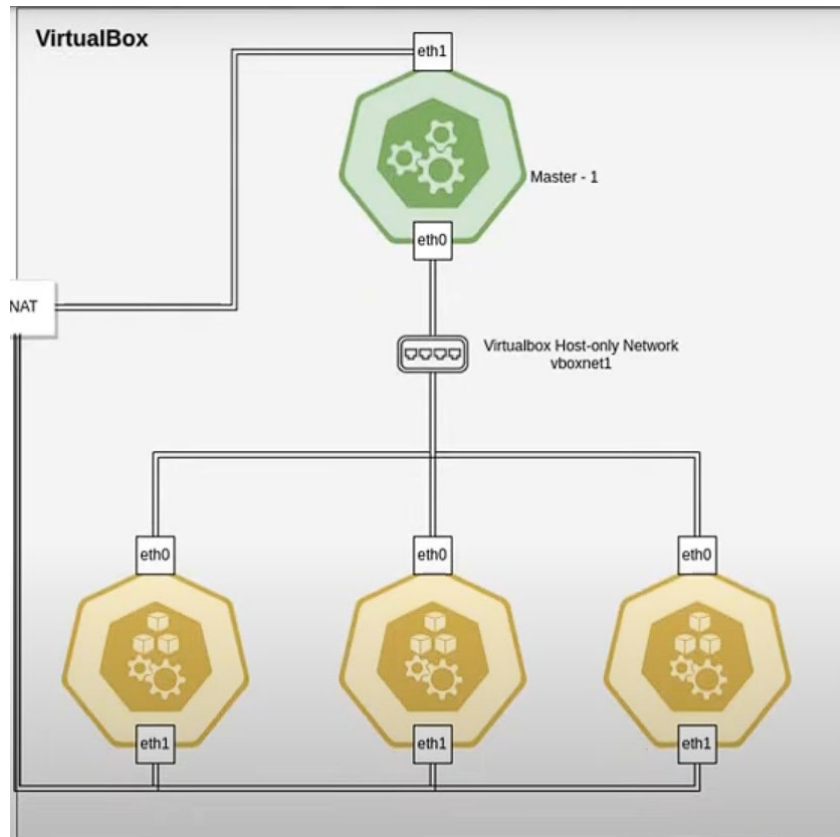
Tuy nhiên do dựa vào DHCP => IP sẽ được cấp động, không cố định

Host-only Networking

Máy host sẽ join vào hệ thống máy ảo => mạng riêng, gần giống Internal Network, nhưng máy Windows gia nhập vào luôn

Mô tả bài toán:

Xây dựng hệ thống máy tính gồm: 1 máy master (2GB RAM) 4 máy worker (1GB Ram) như thiết kế dưới:



(Trong hình vẽ đang thiếu 1 máy worker)

Truy cập thông qua mạng riêng là host-only network, giao thức eth0. Giao tiếp ra ngoài qua eth1.

Câu hỏi: Những máy bên ngoài có thể truy xuất vào bên trong hay không?

Ngoại trừ host, còn lại máy bên ngoài không truy xuất được.

Chuẩn bị:

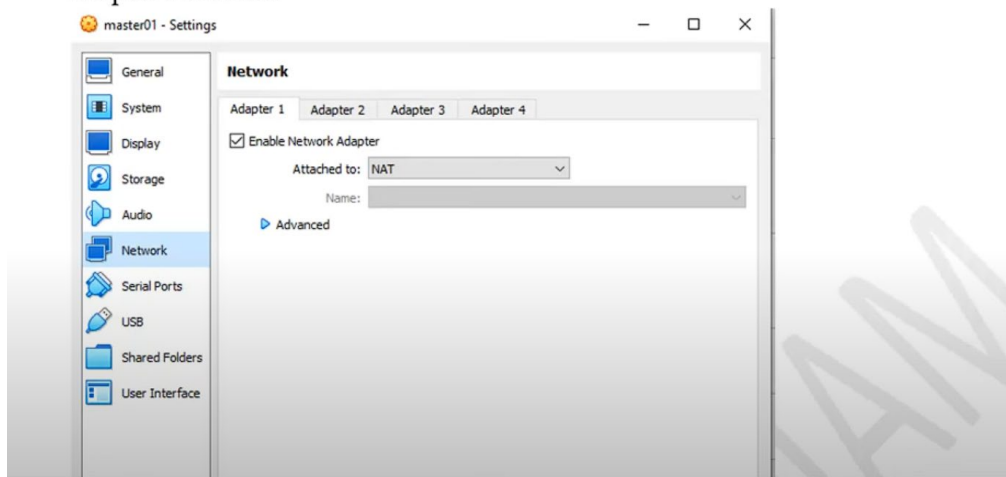
1. Cài ssh (server lẫn client), docker engine
2. Cài thêm gói network manager tools:

```
sudo apt update
```

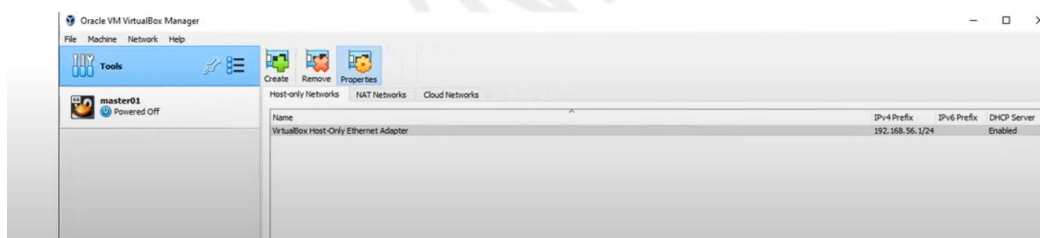
```
sudo apt install ifupdown net-tools openssh-client openssh-server
```
3. Setup 2 adapter: 1 - nat, 2 - host only network

Thực hiện:

1. **Cấu hình mạng NAT cho các máy tính.** Mỗi máy tính ta vào cấu hình network ch adapter 1 như sau



2. Tạo host-only network tự động: Phân IP , các nhóm sẽ tự cấu hình lại phù hợp với từng nhóm. Hoặc thiết lập IP cho host-only adapter như hình (đây là IP của adapter, tương ứng với vboxnet01 trên hình)



Tools -> Host-only network -> tạo mới (create): 192.168.56.1/24

Tạo xong thì vào CMD: ipconfig

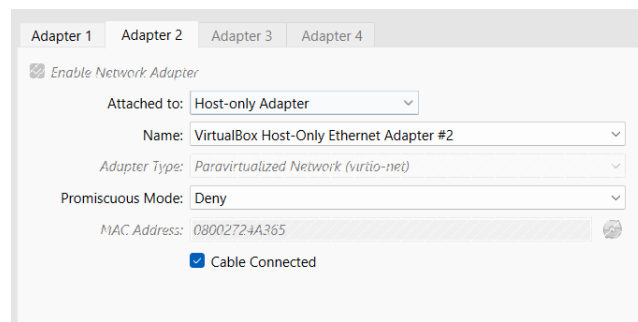
Hoặc dùng DHCP để cấp IP động

[Cập nhật]

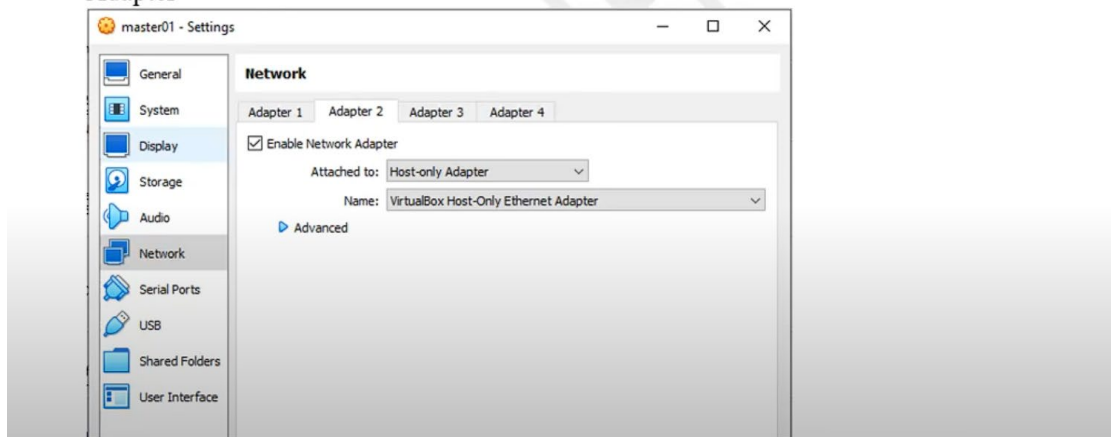
Lỗi: Failed to open/create the internal network '**HostInterfaceNetworking-VirtualBox Host-Only Ethernet Adapter**' (VERR_INTNET_FLT_IF_NOT_FOUND).

Failed to attach the network LUN (VERR_INTNET_FLT_IF_NOT_FOUND).

>> Thử tạo lại 1 adapter mới và config như sau:



3. Với từng máy , chọn cấu hình host-only network cho từng máy phù hợp trong tab adapter 2 như sau: master, work01, worker02, worker03, worker04 → Virtual host-only Ethernet Adapter



Hướng dẫn cấu hình mạng Nat và Host-Only Network cho máy master01

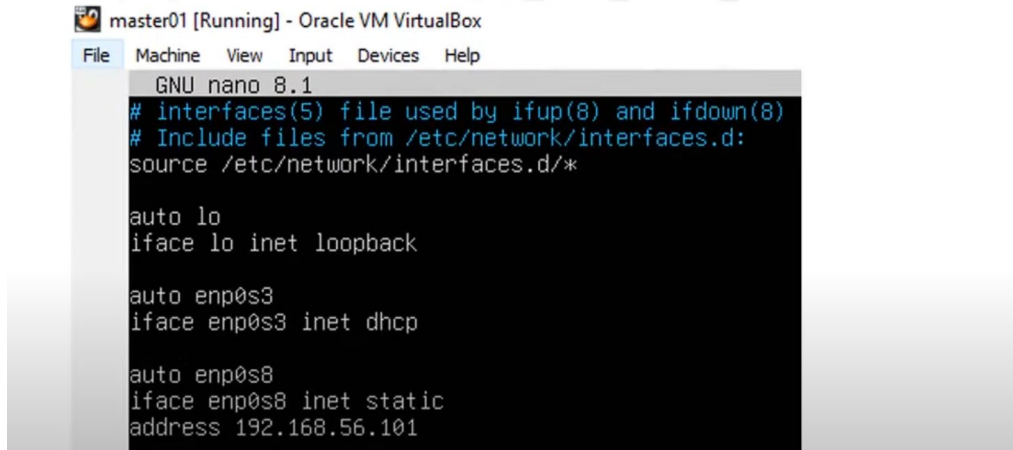
1. Khởi động máy master01
2. Trỏ về thư mục gốc

```
pwd  
cd /  
ls
```

```
osboxes@osboxes:~$ pwd  
/home/osboxes  
osboxes@osboxes:~$ cd /  
osboxes@osboxes:/$ ls  
bin boot cdrom dev etc home lib lib64 lost+found media mnt opt proc root run sbin snap srv  
osboxes@osboxes:/$
```

3. Tiến hành thiết lập IP tĩnh cho mạng host-only network
`sudo nano /etc/network/interfaces`

4. Sau đó thiết lập cho máy `master01` với IP host-only network là `192.168.56.101` . Còn các mạng loop back và NAT thì để auto cấp IP bởi DHCP



```
master01 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 8.1
# interfaces(5) file used by ifup(8) and ifdown(8)
# Include files from /etc/network/interfaces.d:
source /etc/network/interfaces.d/*

auto lo
iface lo inet loopback

auto enp0s3
iface enp0s3 inet dhcp

auto enp0s8
iface enp0s8 inet static
address 192.168.56.101
```

Bấm CTRL + O

Enter sau đó CTRL X

5. Sau đó, khởi động lại máy ảo `master01`
`sudo reboot`
6. Kiểm tra IP mạng: Sử dụng các lệnh
`route -n`
`ip a`
`ifconfig -a`
7. Kiểm tra mạng NAT
ping www.google.com → thấy network reply thành công

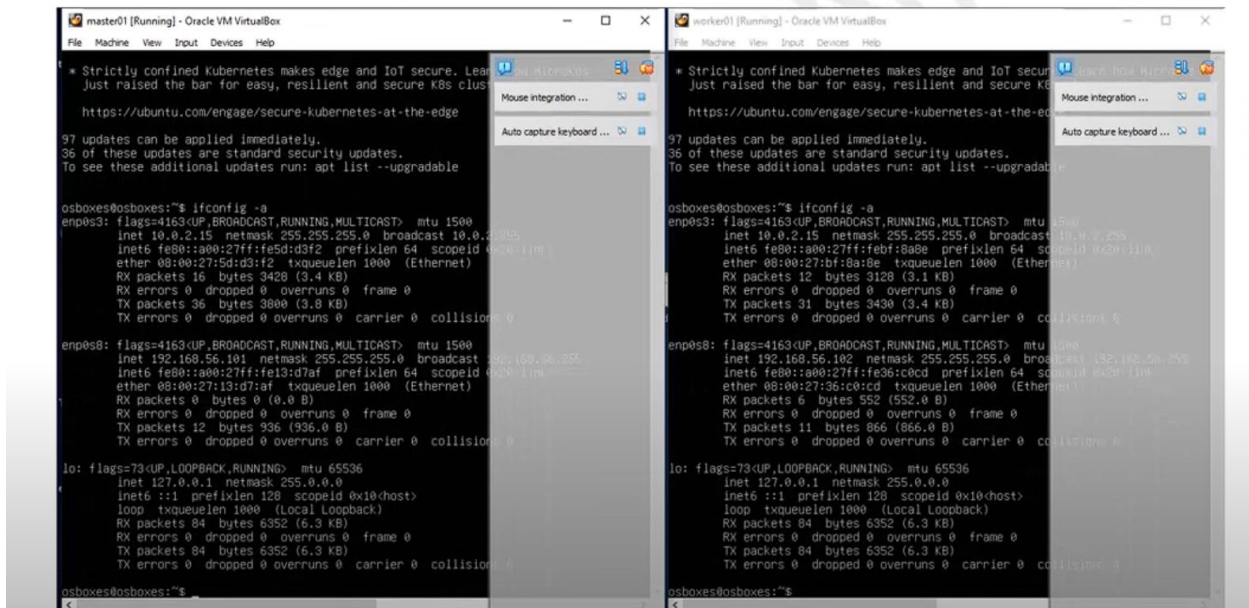
Khởi động chậm do máy đang được cấp phát IP.

8. Kiểm tra mạng Host-only network
- Từ máy windows , mở cmd, sau đó gõ lệnh `ping 192.168.56.101`
 - Hoặc kết nối thử ssh từ máy windows đến máy `master01`
 - o Khởi động ssh server trong máy `master01` và kiểm tra trạng thái là active (running)
 - o Từ máy windows bên ngoài, ta dùng cmd để `ssh osboxes@192.168.56.101`

Tham khảo: <https://sudobird.medium.com/setup-nat-and-host-only-network-on-oracle-vm-virtualbox-9341d7babafe>

(*) Làm tương tự cho 4 máy `worker01`, `worker02`, `worker03`, `worker04`. Phần IP của host-only network tương ứng `192.168.56.102`, `192.168.56.103`, `192.168.56.104`, `192.168.56.105`

Kiểm tra cụm 5 máy tính



Mở 5 máy ảo

1. Tiến hành dùng mỗi máy trong cụm gõ lệnh ping www.google.com
2. Từ các máy windows (host), master01, worker01, worker02, worker03, worker04. Chúng ta có thể ping qua lại nhau thông qua IP từng máy. Hoặc có thể tiến hành ssh đến từng máy master01, worker01, worker02, worker03, worker04 qua lại với nhau:
 - a. Đảm bảo ssh server chạy trên mỗi máy và mỗi máy nên được cài ssh client
 - b. Từ hệ điều hành windows , tiến hành ssh vào mỗi máy
 - c. Từ máy master01 tiến hành ssh vào các máy worker01,02,03,04

Nếu tất cả các test cases đều thành công thì chúng ta đã hoàn tất việc xây dựng hạ tầng mạng ảo cho 5 máy tính (computer group)

Task:

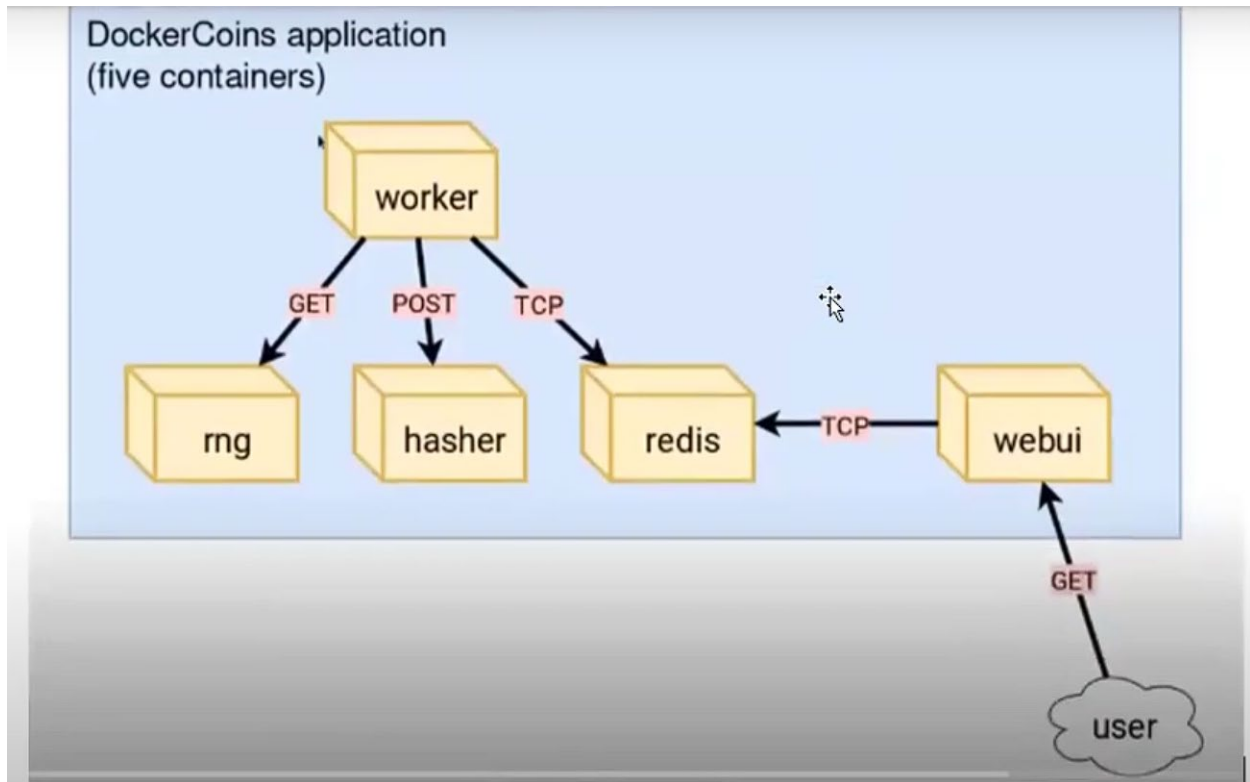
1. Tiến hành vẽ sơ đồ mạng và điền các IP vào phù hợp

>> Config khác của thầy: IP – 192.168.19.10 - 20 để lấy điểm tối đa

2. Sau đó, sẽ chuyển sang xây dựng cụm máy tính Docker Swarm (cluster computer)

Phần 2: Kiểm tra các gói phần mềm trong hệ thống với Docker Engine

Thực hành: Kiểm tra các gói cài đặt phần mềm



Hình 1. Sơ đồ hệ thống các gói phần mềm

Mô tả:

- worker làm việc với rng qua GET, tương tự
- redis là một csdl thuộc dạng memory, worker tương tác với redis qua TCP
- webui: dùng để trực quan các mã hash giống block chain sinh ra trong hệ thống, tương tác với Redis qua TCP.
- User truy cập get vào webui để xem trực quan hệ thống.

Các gói phần mềm:

Nằm trong orchestration-workshop, có folder dockercoins, được cung cấp 4 gói



Redis là một cái image nằm trên Docker Hub.

Trong Docker Compose có config gì:

- Rng nằm trên port 8001
- Hasher P : 8002
- WebUI: 8000
- Redis và worker nằm trong Docker Enginee nên để mặc định.

Test như thế nào?

Pre-condition: Làm xong phần 1

1. Khởi động máy bất kỳ (nên chọn master01), mở CMD và SSH vào máy vừa khởi động.
2. (Dùng cách nào cũng được, lấy mã nguồn về máy ảo)
3. Để ở thư mục người dùng (là usr/ gì đó quên rồi, nhưng mọi người có thể tìm nó, hoặc dùng GitHub clone về tại đường dẫn cd ~)
4. Truy cập vào cái đường dẫn có 4 gói phần mềm

Gõ sudo docker-compose -f docker-compose.yml up

```
sudo docker images
sudo docker ps -a
sudo docker-compose -f docker-compose.yml up
sudo docker-compose -f docker-compose.yml down
sudo docker rm xxx
sudo docker rmi xxx

sudo nano webui.js

sudo docker build -t webui .
sudo docker container run --name webui -p 8000:80 webui

sudo docker stop xxx
sudo docker start xxx

sudo docker image prune -a
sudo docker system prune -a

sudo docker exec -it xxx sh

sudo docker container logs webui

scp -r orchestration-workshop-mrnam osboxes@192.168.56.101:
```

Truy cập thử vào bằng ip của máy master, theo port của các gói phần mềm.

➤ Có thể test thử dùng post get bằng post man để test các gói.

Request sẽ chạy liên tục, mình đang giả lập các request gửi đến server -> Dẫn đến quá tải

Mục tiêu là cho ta thấy điều đó, và đồ án này sẽ giải quyết vấn đề đó.

Để tắt hệ thống, CTRL + C

Sau khi test xong, Gõ `sudo docker-compose -f docker.compose.yml down` để xoá hết container.

Check lại xem còn chạy không bằng `sudo docker ps -a`

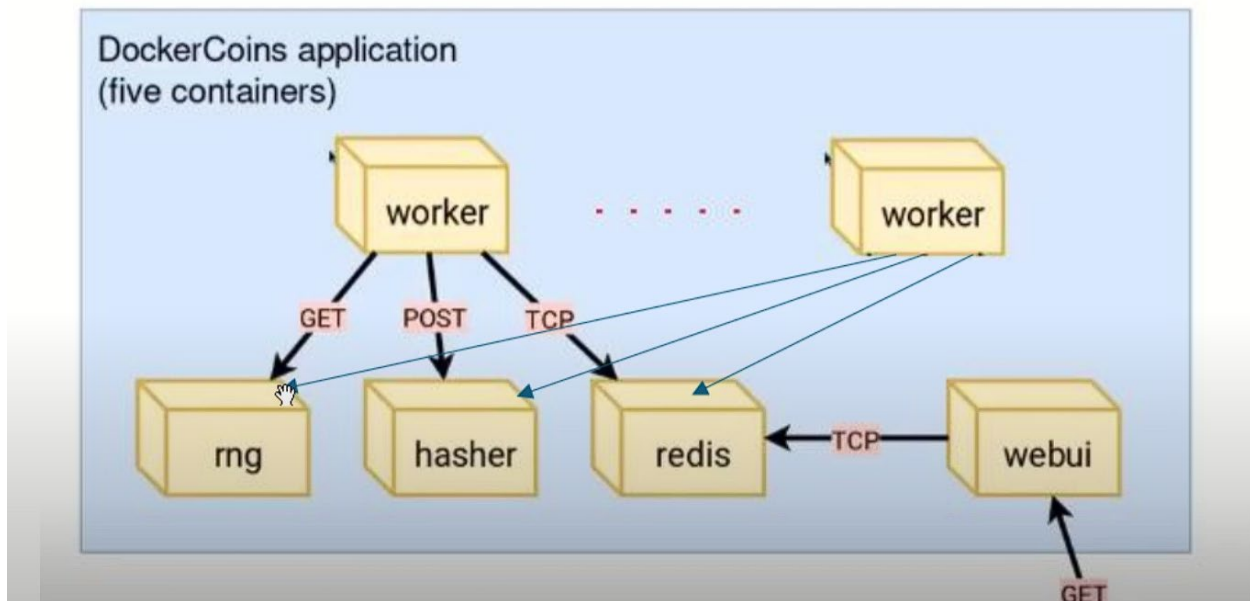
Docker down thì nó sẽ giữ lại file ảnh, xoá thì gõ: `sudo docker image prune -a`

Check lại gõ: `sudo docker images`

Phần 3: Khám phá hiện tượng Bottleneck trong hệ thống Microservices

Thực hành: Khám phá hiện tượng nghẽn cổ chai

Chúng ta sẽ giả lập 10 app workers gọi request đến các dịch vụ rng, hasher, redis. Khi có request tăng đột biến thì dịch vụ sẽ chậm lại, nghẽn cổ chai sẽ xuất hiện.



Giả sử ta tăng cường thêm 10 dịch vụ worker để đào coin (mã hash). Khi đó, số lượng request (yêu cầu) gọi đến rng và hasher là rất lớn → Hệ thống sẽ hoạt động như thế nào?

Định nghĩa đối tượng nghẽn cổ chai (bottleneck): Là dịch vụ mà tại đó độ trễ thời gian phản hồi cao (high latency)

Để khám phá hiện tượng nghẽn cổ chai, ta sẽ hiệu chỉnh RAM của máy master01 lên 4GB

Làm sao để biết nghẽn cổ chai, nghẽn cổ chai là gì?

- Chúng ta phải đặt ra độ trễ thời gian phản hồi
- Khi dịch vụ có thời gian phản hồi trên 01 request quá cao (high latency), dịch vụ đó đang bị bottleneck.

Bắt đầu thực hành:

- Pre-condition: Để đạt hiệu quả cao nhất khi thực hành bài này, khuyến nghị hiệu chỉnh Ram của master01 lên 4GB.
- 1. Khởi động máy master 01 và đăng nhập hệ thống.
- 2. Compose hệ thống và thiết lập các dịch vụ chạy dưới dạng nền (background).

sudo docker-compose -f docker-compose.yml up -d

```
osboxes@osboxes:~/orchestration-workshop-mrnam/dockercoins$ ls
docker-compose.yml hasher rng webui worker
osboxes@osboxes:~/orchestration-workshop-mrnam/dockercoins$ sudo docker-compose -f docker-compose.yml up -d
```

Sau khi compose thì các dịch vụ chạy dạng nền.

```
Creating worker ... done
Creating redis ... done
Creating rng ... done
Creating webui ... done
Creating hasher ... done
```

3. Kiểm tra các app đang chạy trong hệ thống compose

sudo docker-compose ps

```
osboxes@osboxes:~/orchestration-workshop-mrnam/dockercoins$ sudo docker-compose ps
```

Name	Command	State	Ports
hasher	python hasher.py	Up	0.0.0.0:8002->80/tcp,:::8002->80/tcp
redis	docker-entrypoint.sh redis ...	Up	6379/tcp
rng	python rng.py	Up	0.0.0.0:8001->80/tcp,:::8001->80/tcp
webui	node webui.js	Up	0.0.0.0:8000->80/tcp,:::8000->80/tcp
worker	python worker.py	Up	

4. Giám sát các container: Luồng hoá (streaming) container log trên mỗi container để giám sát các container, ở đây là hiện mỗi lần 10 logs:

sudo docker-compose logs --tail 10 --follow

```
osboxes@osboxes:~/orchestration-workshop-mrnam/dockercoins$ sudo docker-compose logs --tail 10 --follow
Attaching to rng, webui, hasher, redis, worker
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:21] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:21] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:21] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:22] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:22] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:22] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:22] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:22] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:22] "POST / HTTP/1.1" 200 -
hasher | 172.19.0.5 - - [19/Feb/2025 09:46:22] "POST / HTTP/1.1" 200 -
```

Bấm Ctrl S để dừng log view
Bấm Ctrl Q để resume log view
Bấm Ctrl C để thoát log view

5. Trên host Windows, vào GUI của app để xem trực quan hoá dữ liệu đào mã hash (mining coin) ¹



¹ **Task:** Tự giải thích, nhận xét và đánh giá số liệu. Tự xem mã hash trên số giây là bao nhiêu.

6. Gõ lệnh *top* để xem CPU và Memory usage: *sudo top* ²

```
osboxes@osboxes:~/orchestration-workshop-mrnam/dockercoins$ sudo top
top - 09:55:59 up 20 min, 3 users, load average: 0.08, 0.22, 0.21
Tasks: 128 total, 1 running, 127 sleeping, 0 stopped, 0 zombie
%Cpu(s): 11.1 us, 0.0 sy, 0.0 ni, 88.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3403.8 total, 2318.3 free, 588.8 used, 725.2 buff/cache
MiB Swap: 4096.0 total, 4096.0 free, 0.0 used, 2815.0 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 8978 root        20   0   9456   5264   3216 R   16.7   0.2   0:00.02 top
 2212 root        20   0  37004  31900  9664 S    8.3   0.9   0:33.08 python
    1 root        20   0  22808  14440 10216 S    0.0   0.4   0:05.89 systemd
    2 root        20   0     0     0     0 S    0.0   0.0   0:00.00 kthreadd
    3 root        20   0     0     0     0 S    0.0   0.0   0:00.00 pool_workqueue_release
    4 root         0 -20     0     0     0 I    0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root         0 -20     0     0     0 I    0.0   0.0   0:00.00 kworker/R-sync_wq
```

7. Gõ *sudo vmstat 3* để xem I/O usage (si/so/bi/bo) ³

```
osboxes@osboxes:~/orchestration-workshop-mrnam/dockercoins$ sudo vmstat 3
procs -----memory----- --swap-- --io---- -system-- -----cpu-----
 r  b   swpd   free   buff   cache   si   so    bi    bo    in   cs us sy id wa st gu
 0  0     0 2373928 43548 699300    0    0   434  178 1382   5  8  3 88  0  0  0
 0  0     0 2373924 43548 699324    0    0     0    0 1437 493  7  2 91  0  0  0
 0  0     0 2373924 43556 699324    0    0     0    9 1363 445  7  1 92  0  0  0
 0  0     0 2373924 43564 699324    0    0     0    9 1340 457  8  1 91  0  0  0
 0  0     0 2373924 43564 699332    0    0     0    0 1360 475  8  1 91  0  0  0
 0  0     0 2373924 43572 699340    0    0     0    5 1347 467  7  2 91  0  0  0
 0  0     0 2373924 43572 699344    0    0     0   19 1382 536  8  2 90  0  0  0
```

Lúc này hệ thống chỉ có 5 app chạy: worker, rng, hasher, redis và webui.

Focus vào 3 dịch vụ worker, rng, hasher.

Giờ ta sẽ tăng lên 10 worker => Số lượng request đến các dịch vụ rng, hasher sẽ được tăng lên 10 lần.

8. Tiến hành scale số lượng app worker lên thành 10 app workers.

sudo docker-compose scale worker=10

hoặc *sudo docker-compose --scale worker=10*

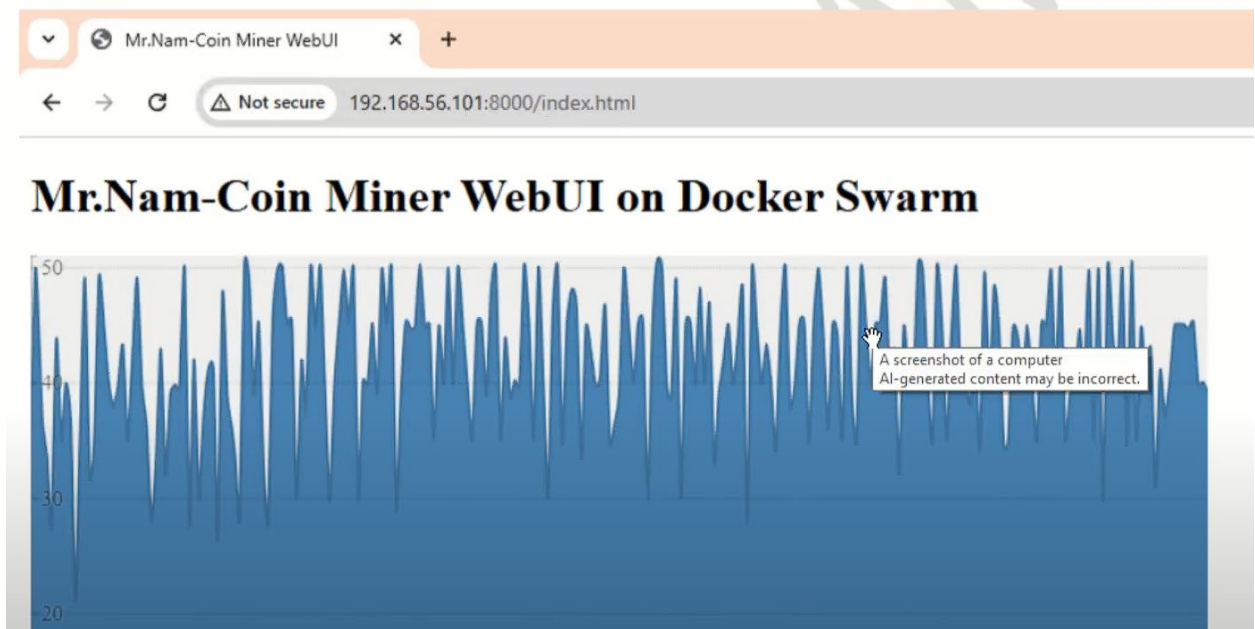
```
osboxes@osboxes:~/orchestration-workshop-mrnam/dockercoins$ sudo docker-compose scale w
WARNING: The scale command is deprecated. Use the up command with the --scale flag inst
Creating dockercoins_worker_2 ... done
Creating dockercoins_worker_3 ... done
Creating dockercoins_worker_4 ... done
Creating dockercoins_worker_5 ... done
Creating dockercoins_worker_6 ... done
Creating dockercoins_worker_7 ... done
Creating dockercoins_worker_8 ... done
Creating dockercoins_worker_9 ... done
Creating dockercoins_worker_10 ... done
```

Khi tăng số lượng worker lên 10, tự đặt ra những câu hỏi: đánh giá io usage, cpu/memory used ở các bước trước,...

² **Task:** Nhìn vào số liệu, tự viết, giải thích, nhận xét và đánh giá số liệu.

³ **Task:** Nhìn vào số liệu, tự viết, giải thích, nhận xét và đánh giá số liệu.

9. **Câu hỏi:** Điều gì sẽ xảy ra? (Gợi ý: Số lượng request gọi đến dịch vụ rng, hasher, tỉ lệ coin được đào tăng lên bao nhiêu lần, thời gian phản hồi của 2 dịch vụ rng và hasher,).⁴



Current mining speed sẽ tăng lên.

10. **Câu hỏi:** Khám phá latency của 2 dịch vụ rng và hasher

Tại sao 2 dịch vụ này sẽ được chú ý để khám phá độ trễ (gợi ý: chú ý số lượng request mà hai dịch vụ này phải xử lý khi tăng số lượng worker=10 so với ban đầu là 1 worker).\

Hướng dẫn đo lường:

- a. Để đo được, ta cần cài: tool httping để đo lường latency của dịch vụ.

```
osboxes@osboxes:~/orchestration-workshop-mrnam/dockercoins$ sudo apt-get install httping
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

- b. Đo lường độ trễ phản hồi của các dịch vụ

- Rng service:

- `sudo httping -c 10 localhost:8001`
- Gợi ý: Đo lường, ghi trung bình thời gian phản hồi, có thể lấy 10 mẫu, sau đó chia trung bình ra rồi ghi vào.

- Hasher service:

- `sudo httping -c 10 localhost:8001`
- Gợi ý: Đo lường, ghi trung bình thời gian phản hồi, có thể lấy 10 mẫu, sau đó chia trung bình ra rồi ghi vào.

⁴ **Task:** Nhìn vào số liệu, tự viết, giải thích, nhận xét và đánh giá số liệu.

11. Sinh viên tự xác định hiện tượng nghẽn cổ chai xảy ra ở dịch vụ nào?

Gợi ý: So sánh time response của rng và hasher

12. Sau khi xác định được vấn đề nghẽn cổ chai ở dịch vụ nào, ta có thể làm sạch hệ thống và chuẩn bị thiết kế giải pháp tối ưu hoá hệ thống bằng lệnh:

```
sudo docker-compose -f docker.compose.yml down
```

```
sudo docker ps -a
```

```
sudo docker image prune -a
```

```
sudo docker images
```

Phần 4: Xây dựng cụm máy tính dựa trên Docker Swarm

Thực hành: Khám phá Docker Swarm (Cụm máy docker)

- **Cụm máy Docker** là tập hợp các máy chủ có cài đặt Docker Engine. Nếu **Docker Engine** cho phép *triển khai nhiều ứng dụng trên một mạng* một cách đơn giản, thì **cụm máy Docker** giúp các máy chủ kết hợp với nhau để *tối ưu hóa và vận hành hệ thống* trong kiến trúc Microservice, đặc biệt là trong các hệ thống phần mềm lớn.
- Máy master (hay là gọi manager) trong hệ thống docker swam.

1. Docker Engine phiên bản mới hỗ trợ và tích hợp Swarmkit (câu hỏi: Swarmkit là gì?)
Chọn máy master01 để thực hiện cấu hình trở thành máy master trong hệ thống docker swarm, theo mô hình master-slave. Đăng nhập và ssh vào máy master01 từ máy host windows. Làm quen các lệnh CLI trong docker swarm:

sudo docker swarm

sudo docker node

sudo docker service

```
osboxes@osboxes:~$ sudo docker swarm
[sudo] password for osboxes:

Usage:  docker swarm COMMAND

Manage Swarm

Commands:
  init      Initialize a swarm
  join      Join a swarm as a node and/or manager

Run 'docker swarm COMMAND --help' for more information on a command.
osboxes@osboxes:~$ sudo docker node
```

```
Usage:  docker node COMMAND

Manage Swarm nodes

Commands:
  demote    Demote one or more nodes from manager in the swarm
  inspect   Display detailed information on one or more nodes
  ls        List nodes in the swarm
```

```
Run 'docker node COMMAND --help' for more information on a command.
osboxes@osboxes:~$ sudo docker service
```

```
Usage:  docker service COMMAND

Manage Swarm services

Commands:
  create    Create a new service
  inspect   Display detailed information on one or more services
  logs      Fetch the logs of a service or task
  ls        List services
  ps        List the tasks of one or more services
```

Câu hỏi: Swarm là gì?

2. Chế độ Swarm (Swarm mode), mặc định thì chế độ này không hoạt động (inactive). Ta có thể thử nghiệm

`sudo docker node ls`

```
osboxes@osboxes:~$ sudo docker node ls
Error response from daemon: This node is not a swarm manager. Use "docker swarm init" or "docker swarm join" to connect this node to swarm and try again.
```

3. Kích hoạt chế độ Swarm mode bằng cách khởi tạo cụm máy docker (docker swarm)

`sudo docker swarm init`

```
osboxes@osboxes:~$ sudo docker swarm init
Error response from daemon: could not choose an IP address to advertise since this system has multiple addresses on different interfaces (10.0.2.15 on enp0s3 and 192.168.56.101 on enp0s8) - specify one with --advertise-addr
```

<Tự dịch lỗi, giải thích lỗi gì đang xảy ra cho (2), (3)>

Gợi ý: Lỗi liên quan tới network do đang có 2 adapter network (host-only với nat)

4. Chọn mạng host-only network để xây dựng cụm máy tính (swarm cluster) với cổng mặc định trên từng node dùng giao tiếp giữa các node trong swarm là 2377, ta có thể không cần ghi 2377 trong câu lệnh. Câu lệnh bên dưới khởi động tạo docker swarm với máy master01 sẽ trở thành máy có vai trò quản lý (master)

```
sudo docker swarm init --advertise-addr 192.168.56.101:2377
```

```
sudo docker swarm init --advertise-addr 192.168.56.101:2377
```

```
osboxes@osboxes:~$ sudo docker swarm init --advertise-addr 192.168.56.101:2377
Swarm initialized: current node (m8we3e4b19ua0uia4u75nurpe) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-67kk2eectciavertysk84q0wd30d5kquvvkn8trfe4qa92rt-ct05v0uspn66o1y5wr8zrbyjh 192.168.56.101:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

<Sinh viên giải thích giải thích tại sao không dùng mạng NAT để xây dựng cụm docker>

Sinh viên giải thích 2 mã token sinh ra này mục đích làm gì ? Nhớ lưu lại để sử dụng cho phần “Thêm máy worker vào cụm máy docker”

```
docker swarm join --token SWMTKN-1-67kk2eectciavertysk84q0wd30d5kquvvkn8trfe4qa92rt-ct05v0uspn66o1y5wr8zrbyjh 192.168.56.101:2377
```

Câu hỏi: Tại sao không dùng NAT để xây dựng cụm docker

Câu hỏi: Hai mã token sinh ra nhằm mục đích gì?

Gợi ý: Sinh viên đọc hiểu “Token Generation”

5. Kiểm tra xem Swarm mode đã hoạt động

`sudo docker info`

```
osboxes@osboxes:~$ sudo docker info
[sudo] password for osboxes:
Client: Docker Engine - Community
Version: 27.5.1
Context: default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version: v0.20.0
    Path: /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version: v2.32.4
    Path: /usr/libexec/docker/cli-plugins/docker-compose
```

```
Server:
Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
Images: 0
Server Version: 27.5.1
```

Xem mục Nodes sẽ thấy **Nodes: 1**

Ø Nodes: 1 (master01, là đang có 1 node -> manager)

Ø **Câu hỏi: Xem log xem nodes là 0 hay là 1**

6. Kiểm tra trạng thái của node trong docker swarm hiện tại là máy master01 với vai trò là LEADER (máy master trong mô hình master-slave)

`sudo docker node ls`

```
osboxes@osboxes:~$ sudo docker node ls
[sudo] password for osboxes:
ID                HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
m8we3e4b19ua0uia4u75nurpe *  osboxes          Ready     Active           Leader             27.5.1
```

7. Như vậy máy master01 đã trở thành manager (master hay leader) trong cụm máy docker (docker swarm) được tạo. Bây giờ ta tiến hành thêm một node khác (với vai trò là worker hay còn gọi là slave) vào hệ thống cụm máy (docker swarm) đã được tạo.

Để tiến hành thêm node (như là worker) vào cụm máy docker thì ta cần sử dụng token được sinh ra ở mục trước đó.

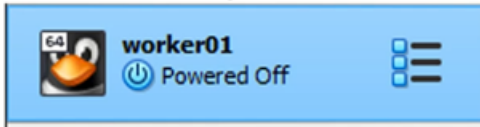
Ta có thể hiển thị lại mã token đã sinh ra

`sudo docker swarm join-token worker`

```
osboxes@osboxes:~$ sudo docker swarm join-token worker
To add a worker to this swarm, run the following command:

docker swarm join --token SWMTKN-1-67kk2euctiaxvertysk84q0wd30d5skuvvkn8trfe4qa92rt-ct05v0uspn66o1y5wr8zrbyjh 192.168.56.101:2377
```

8. Tiến hành bật máy worker01



- Kiểm tra trong máy worker01 đã chạy dịch vụ ssh server hay chưa
- Từ máy host windows , mở thêm một giao diện command promt để ssh vào máy worker01. Lưu ý: Lúc này cần phân biệt cửa sổ command promt nào đang thao tác với máy nào để tránh nhầm lẫn.

```
osboxes@192.168.56.101's password:
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-14-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Feb 20 03:15:17 PM UTC 2025

System load:  0.0      Processes:      112
Usage of /:   8.1% of 97.87GB   Users logged in: 0
Memory usage: 7%      IPv4 address for enp0s3: 10.0.2.15
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.
https://ubuntu.com/engage/secure-kubernetes-at-the-edge

66 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Last login: Thu Feb 20 15:02:28 2025 from 192.168.56.1
osboxes@osboxes:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5d:d3:f2 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 85112sec preferred_lft 74312sec
    inet6 fe80::a00:27:5d:d3:f2/64 scope link proto kernel ll
        valid_lft forever preferred_lft forever

The authenticity of host '192.168.56.102 (192.168.56.102)' can't be established.
ED25519 key fingerprint is SHA256:3LC+hjFuI7GRd9ChgAnyy084SymlicQqDyYBFVmcDg.
This host key is known by the following other names/addresses:
  C:\Users\Admin/.ssh/known_hosts:24: [127.0.0.1]:2222
  C:\Users\Admin/.ssh/known_hosts:27: 192.168.56.101
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.102' (ED25519) to the list of known hosts.
osboxes@192.168.56.102's password:
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-9-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Feb 20 03:23:01 PM UTC 2025

System load:  0.06     Processes:      113
Usage of /:   7.2% of 97.87GB   Users logged in: 0
Memory usage: 19%      IPv4 address for enp0s3: 10.0.2.15
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.
https://ubuntu.com/engage/secure-kubernetes-at-the-edge

97 updates can be applied immediately.
36 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sat Jan 25 13:36:19 2025 from 192.168.56.101
osboxes@osboxes:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

Để ý xem đang connect với máy nào để tránh nhầm lẫn.

- Copy đoạn mã

```
docker swarm join --token SWMTKN-1-67kk2eucitaxvertysk84q0wd30d5kquvvkn8trfe4qa92rt-ct05v0uspn66o1y5wr8zrbyjh 192.168.56.101:2377
```

và dán vào cửa sổ ssh của máy worker01 để thực thi câu lệnh chạy trên máy worker01

```
osboxes@osboxes:~$ sudo docker swarm join --token SWMTKN-1-67kk2eucitaxvertysk84q0wd30d5kquvvkn8trfe4qa92rt-ct05v0uspn66o1y5wr8zrbyjh 192.168.56.101:2377
This node joined a swarm as a worker.
```

- Quay trở lại cửa sổ giao diện ssh của máy master01 và xem thông tin danh sách các node trong hệ thống swarm.

sudo docker info

```
13 Managers: 1
ClusterID: mc46si0rvvgq640qws88rcm1dc
Managers: 1
Nodes: 2
Default Address Pool: 10.0.0.0/8
SubnetSize: 24
```

- Hiện thị thông tin danh sách các nodes trong cụm docker (docker swarm)

sudo docker node ls

```
osboxes@osboxes:~$ sudo docker node ls
ID                HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
f72xul4iuhha4x9x3edpv3og1  osboxes          Ready     Active
m8we3e4b19ua0uia4u75nurpe *  osboxes          Ready     Active            Leader             27.5.1
```

Dấu (*) thể hiện node manager

9. Sinh viên làm tương tự để thêm các máy worker vào swarm cluster

10. Một vài câu lệnh quản trị các node trên swarm

- a. Một node worker muốn rời khỏi swarm
 - i. Thao tác trên máy worker: `docker swarm leave`
- b. Node manager muốn rời khỏi swarm
 - i. Thao tác trên máy manager: `sudo docker swarm leave --force`
- c. Manager node muốn xóa một worker node ra khỏi docker swarm
 - i. Thao tác trên máy manager: `sudo docker node rm <tên worker node>`

(Để ý 10, sẽ có những trường hợp thao tác trên máy khác nhau, khi nào dùng trên manager, khi nào dùng trên worker)

[Đọc thêm] SwarmKit là gì?

Docker Swarm là một công cụ điều phối Docker nguyên bản ra đời từ năm 2014. Nó hoạt động độc lập với Docker Engine và được sử dụng để kết nối các Docker Engine lại với nhau nhằm tạo thành một cụm (cluster). Sau đó, ta có thể kết nối với Swarm và chạy các container trên cụm này. Swarm có một số tính năng như:

- Cho phép chỉ định một dịch vụ khám phá (discovery service).
- Có một số cơ chế kiểm soát vị trí đặt container (sử dụng bộ lọc, ràng buộc, chiến lược phân phối, v.v.).
- Cung cấp cùng một API như Docker Engine, giúp các công cụ bên thứ ba có thể tương tác một cách liền mạch.

SwarmKit là một công cụ mới (2016) do đội ngũ Docker phát triển, cung cấp chức năng chạy cụm và phân phối tác vụ đến các máy trong cụm. Các tính năng chính của SwarmKit bao gồm:

- **Phân tán:** SwarmKit sử dụng Thuật toán Đồng thuận Raft để điều phối và không phụ thuộc vào một điểm lỗi duy nhất để ra quyết định.
- **Bảo mật:** Giao tiếp giữa các node và tư cách thành viên trong Swarm được bảo mật ngay từ đầu. SwarmKit sử dụng TLS hai chiều để xác thực node, ủy quyền vai trò và mã hóa dữ liệu truyền tải, đồng thời tự động cấp phát và luân chuyển chứng chỉ.
- **Đơn giản:** SwarmKit dễ vận hành và giảm thiểu sự phụ thuộc vào hạ tầng. Nó không cần cơ sở dữ liệu bên ngoài để hoạt động.

Docker Swarm Mode (từ phiên bản 1.12 trở đi) sử dụng các thư viện và chức năng của SwarmKit để giúp điều phối container trên nhiều máy chủ trở nên đơn giản và an toàn hơn. Một bộ tính năng mới (đáng chú ý nhất là **docker swarm**) đã được tích hợp trực tiếp vào Docker, cho phép chúng ta khởi tạo Swarm mới và triển khai các tác vụ lên cụm.

Docker Swarm không bị khai tử mà vẫn là một phương pháp khả thi để điều phối Docker trên nhiều máy chủ. Tuy nhiên, **Docker Swarm Mode** (sử dụng thư viện SwarmKit bên trong) là cách được khuyến nghị khi bắt đầu một dự án Docker mới có yêu cầu điều phối trên nhiều máy chủ.

Một trong những tính năng quan trọng trong Docker 1.12 là **Swarm Mode**. Trước đó, Docker đã hỗ trợ Swarm để điều phối container từ phiên bản 1.6. Docker cũng đã phát hành **SwarmKit** dưới dạng một dự án mã nguồn mở chuyên dùng để điều phối hệ thống phân tán chỉ vài tuần trước khi Docker 1.12 (RC) ra mắt.

- **Swarm** đề cập đến chức năng Swarm truyền thống.
- **Swarm Mode** là chế độ Swarm mới được thêm vào từ phiên bản 1.12.
- **SwarmKit** là dự án mã nguồn mở cung cấp các thành phần điều phối bên trong.

Đọc chi tiết hơn tại: <https://dockerlabs.collabnix.com/intermediate/swarm/difference-between-docker-swarm-vs-swarm-mode-vs-swarmkit.html>

[Đọc thêm] Tạo Token (Token Generation)

- Khi chạy lệnh `docker swarm init`, ta nhận được thông báo xác nhận rằng **node hiện tại đã trở thành manager**:

> *Swarm initialized: current node (8jud...) is now a manager.*

- Docker tự động tạo **hai security token** (giống như mật khẩu) để bảo vệ cụm máy.
- CLI sẽ hiển thị **lệnh** để thêm các node khác vào cụm, sử dụng **token của worker**.

Ví dụ, để thêm một worker vào swarm, chạy lệnh sau trên node mới:

```
docker swarm join \
```

```
--token SWMTKN-1-59fl4ak4nqjmao1of... \
```

```
172.31.4.182:2377
```

Phần 5: HIGH AVAILABILITY & AUTO SCALING & LOAD BALANCING IN DOCKER SWARM

Thực hành: Quản trị và vận hành cụm máy docker

AUTO SCALING, AUTO LOAD BALANCING AND HIGH AVAILABILITY IN DOCKER SWARM

1. Kiểm tra lại hệ thống docker swarm hiện có: 1 máy có vai trò là manager (master01) và 4 máy còn lại có vai trò worker. Ở giao diện thao tác của máy manager, ta liệt kê danh sách các node trong swarm

sudo docker node ls

```
osboxes@osboxes:~$ sudo docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
f72xu14iuhha4x9x3edpv3og1  osboxes        Ready    Active
m8we3e4b19ua0uia4u75nurpe *  osboxes        Ready    Active          Leader            27.5.1
```

M8we3e4b... có một dấu *, nó chính là máy đang thao tác

Các máy đều có hostname là osboxes -> Để dễ nhận biết hơn, ta có thể thực hiện đổi tên hostname:

- sudo hostname
- sudo hostnamectl set-hostname <tên cần set> (Gợi ý của thầy set theo vpsx – x là số thứ tự tăng dần).
- sudo hostname
- reboot

(*) Lưu ý: ta chỉ đổi tên hostname alias hiển thị và sử dụng trên hệ thống mạng, nên mọi thứ khác còn lại trên máy tính ban đầu không thay đổi như: tên tài khoản đăng nhập, mật khẩu đăng nhập, thông số đăng nhập từ ssh (ví dụ: ssh [osboxes@192.168.56.101](#))

3. Thao tác tại giao diện máy master01 (vai trò manager), tiến hành liệt kê lại các máy tính trong cụm docker

```
osboxes@vps0:~$ sudo docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
m8we3e4b19ua0uia4u75nurpe *  vps0           Ready    Active          Leader            27.5.1
f72xu14iuhha4x9x3edpv3og1    vps1           Ready    Active
```

Hiện đang có 5 máy (1 manager và 4 worker)

4. Thêm vps1 tham gia vai trò là manager trong cụm. Thao tác ở giao diện ssh tương tác với máy vps0 (hiện đang là manager trong cụm).

`sudo docker node ls`

`sudo docker node promote vps1`

(*) có thể dùng ID thay cho vps1

```
osboxes@vps0:~$ sudo docker node promote vps1
Node vps1 promoted to a manager in the swarm.
osboxes@vps0:~$ sudo docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
m8we3e4b19ua0uia4u75nurpe *	vps0	Ready	Active	Leader	27.5.1
f72xul4iuhha4x9x3edpv3og1	vps1	Ready	Active	Reachable	27.5.1

Từ giao diện thao tác ssh của vps1, ta đã có thể quản lý swarm

Để nâng cấp thêm 1 manager nữa, ta cần vào máy master, sau đó gõ

`sudo docker node promote <hostname/id>`

Từ giờ thì <hostname/id> đã trở thành manager. Ta có thể quản lý swarm từ đó.

Để hạ cấp, thay promote bằng demote (dùng master01/nhưng nếu worker là manager thì sao ta?)

```
osboxes@vps1:~$ sudo docker node ls
[sudo] password for osboxes:
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
m8we3e4b19ua0uia4u75nurpe	vps0	Ready	Active	Leader	27.5.1
f72xul4iuhha4x9x3edpv3og1 *	vps1	Ready	Active	Reachable	27.5.1

Nếu muốn một máy đang ở vai trò manager trở về vai trò worker thì ta có thể sử dụng lệnh demote. Ở giao diện ssh của máy master01 ta gõ

`sudo docker node demote vps1`

```
osboxes@vps0:~$ sudo docker node demote vps1
Manager vps1 demoted in the swarm.
osboxes@vps0:~$ sudo docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
m8we3e4b19ua0uia4u75nurpe *	vps0	Ready	Active	Leader	27.5.1
f72xul4iuhha4x9x3edpv3og1	vps1	Ready	Active		27.5.1

5. Sinh viên tiến hành chọn vps1 và vps2 để nâng cấp thành vai trò manager trong cụm docker.

Câu hỏi đặt ra:

- Hiện tượng single-point-failure xảy ra khi nào?
- Giải pháp nào cho hiện tượng single-point-failure?
- Với hệ thống (high availability – HA) có khả năng chịu được N-point-failure thì ta sẽ cần thiết kế bao nhiêu node giữ vai trò manager?
- Khi trong hệ thống cụm docker swarm có nhiều máy với vai trò là manager thì số lượng các máy có vai trò là manager trong cụm phải hoạt động (active) với tỉ lệ số lượng là bao nhiêu để có thể thao tác quản trị trên cụm? Cho biết lý do? Gợi ý hình bên dưới

```
Error response from daemon: rpc error: code = Unknown desc = The swarm does not have a leader. It's possible that too few managers are online. Make sure more than half of the managers are online.
```

Ở câu hỏi 1, nếu một máy manager bị sập thì cả cụm bị sập luôn. Để giải quyết thì ta sẽ gán nhiều node hơn là máy manager

Câu hỏi 2. Khi máy bị rớt/treo/sập, thì hệ thống vẫn hoạt động nhờ vào các máy còn lại chia sẻ.

Vậy hệ thống có thể chịu được n máy bị sập, thì chúng ta cần bao nhiêu máy manager?

- Chúng ta nên xây dựng $2*n+1$ máy manager.
- Khi ta khởi động 1 cụm máy tính có $2n + 1$ manager, nếu có ít hơn $n+1$ máy với vai trò là manager ở trên cụm k hoạt động thì cụm sẽ k hoạt động được

6. Từ một node có vai trò là manager có mã nguồn orchestration-workshop-mrnam, ở đây ta sử dụng vps0. Ta thao tác kiểm tra hiện trạng hệ thống

```
osboxes@vps0:~$ ls
orchestration-workshop-mrnam
osboxes@vps0:~$ sudo docker container ls
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
osboxes@vps0:~$ sudo docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
```

(master01 đang chứa mã nguồn)

7. Tiến hành build image cho các gói dịch vụ: hasher, worker, rng và webui

- Build image gói rng

- Chuyển thư mục hiện hành vào rng: `cd rng/`
- Tiến hành build image rng: `sudo docker build -t rng .`

Build thành công:

```
osboxes@vps0:~/orchestration-workshop-mrnam/dockercoins/rng$ sudo docker build -t rng .
[+] Building 6.4s (8/8) FINISHED
-> [internal] load build definition from Dockerfile
-> => transferring dockerfile: 127B
-> [internal] load metadata for docker.io/library/python:alpine
-> [internal] load .dockerignore
-> => transferring context: 2B
-> [1/3] FROM docker.io/library/python:alpine@sha256:323a717dc4a010fee21e3f1aac738ee10bb485de4e7593ce242b36ee48d6b352
-> [internal] load build context
-> => transferring context: 28B
-> CACHED [2/3] RUN pip install Flask
-> CACHED [3/3] COPY rng.py /
-> exporting to image
-> => exporting layers
-> => writing image sha256:4f836d6d6ac54c57533261cfcc8586224cc76ed2cf6d2ad63dd294f4ee8c70bb
-> => naming to docker.io/library/rng
osboxes@vps0:~/orchestration-workshop-mrnam/dockercoins/rng$ sudo docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
rng                  latest             4f836d6d6ac5       4 days ago         58.7MB
osboxes@vps0:~/orchestration-workshop-mrnam/dockercoins/rng$
```

`sudo docker image ls`

Sinh viên tự làm tương tự cho các gói còn lại.

```
osboxes@vps0:~/orchestration-workshop-mrnam/dockercoins$ sudo docker image
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
worker              latest             e5e6c9fdc6a7       4 days ago         59.7MB
webui                latest             8144faa4429a       4 days ago         221MB
hasher               latest             c49e0b0a316d       4 days ago         58.7MB
rng                  latest             4f836d6d6ac5       4 days ago         58.7MB
```


- . Từ máy vps0, tiến hành publish các image lên Docker Hub (Public Cloud Docker Registry)
 - a. Đăng nhập Docker Hub
sudo docker login
sau đó, copy mã và mở trình duyệt theo đường link hiển thị:
<https://login.docker.com/activate> , tiến hành dán mã vào hộp thoại

```
osboxes@vps0:~$ sudo docker login
[sudo] password for osboxes:

USING WEB-BASED LOGIN
To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: [REDACTED]
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate
Waiting for authentication in the browser...
```

Đăng nhập thành công

```
WARNING! Your password will be stored unencrypted in /root/.docker/config.json
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores
Login Succeeded
```

Tag (đánh dấu) image rng đã tạo trong máy local lên Docker Hub Registry. Sau đó, tiến hành push image rng lên docker hub

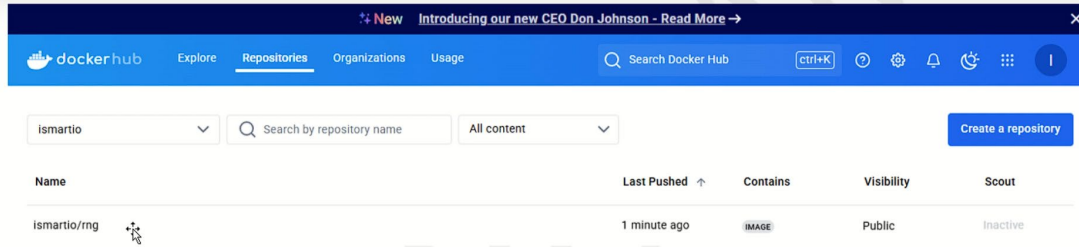
```
sudo docker tag rng ismartio/rng:coinswarm
sudo docker push ismartio/rng:coinswarm
```

VD: đẩy rng lên docker hub vd: ismartio là tên tài khoản. tag là rng

```
osboxes@vps0:~$ sudo docker tag rng ismartio/rng:coinswarm
osboxes@vps0:~$ sudo docker push ismartio/rng:coinswarm
The push refers to repository [docker.io/ismartio/rng]
da2be20403ed: Pushed
bdd87a307275: Pushed
336e6a290569: Mounted from library/python
53d9c097c68d: Mounted from library/python
052b772c7a04: Mounted from library/python
08000c18d16d: Mounted from library/python
coinswarm: digest: sha256:09181ee2329ab962baaa62e52f69fcfda56ae875e8f6de107fa01e22f0f4a8 size: 1575
```

I

Kiểm tra xem image đã được publishing thành công trên Docker Hub



Tương tự làm cho các image hasher, worker, webui với cấu hình tương tự

```
sudo docker tag hasher ismartio/hasher:coinswarm  
sudo docker push ismartio/hasher:coinswarm
```

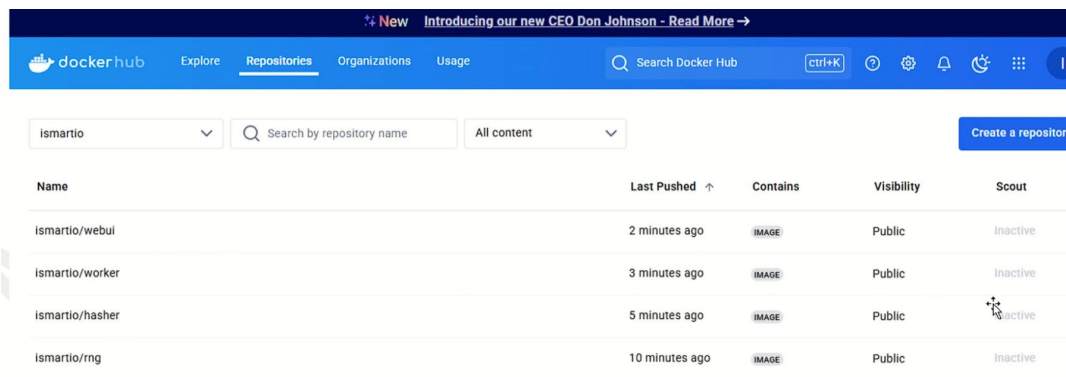
```
sudo docker tag worker ismartio/worker:coinswarm  
sudo docker push ismartio/worker:coinswarm
```

```
sudo docker tag webui ismartio/webui:coinswarm  
sudo docker push ismartio/webui:coinswarm
```

```
osboxes@vps0:~$ sudo docker tag hasher ismartio/hasher:coinswarm  
osboxes@vps0:~$ sudo docker push ismartio/hasher:coinswarm  
The push refers to repository [docker.io/ismartio/hasher]  
7343c725520a: Pushed  
bdd87a307275: Mounted from ismartio/rng  
336e6a290569: Mounted from ismartio/rng  
53d9c097c68d: Mounted from ismartio/rng  
052b772c7a04: Mounted from ismartio/rng  
0800c18d16d: Mounted from ismartio/rng  
coinswarm: digest: sha256:42935f876fe1a0b1c7a2e25ff3f36a1cc186673e3e87aab139913c4dd5109eaa size: 15  
osboxes@vps0:~$ sudo docker tag worker ismartio/worker:coinswarm  
osboxes@vps0:~$ sudo docker push ismartio/worker:coinswarm  
The push refers to repository [docker.io/ismartio/worker]  
b7ca0f7d6d85: Pushed  
31c6ebdd08e7: Pushed  
422caf65abcc: Pushed  
336e6a290569: Mounted from ismartio/hasher  
53d9c097c68d: Mounted from ismartio/hasher  
052b772c7a04: Mounted from ismartio/hasher
```

Mounted là sử dụng lại

Push thành công



Lí do up lên vì đơn giản, local thì không có ra ngoài được

9. Tạo overlay network trong hệ thống swarm

a. Tạo overlay network không mã hóa

```
sudo docker network create --scope=swarm --attachable -d overlay coinswarmnet
```

b. Tạo overlay network có mã hóa

Overlay giúp cho ứng dụng nằm trên nhiều máy giao tiếp được với nhau -> hệ thống mạng nằm trên docker swarm là overlay network.

Tạo overlay network mã hoá

```
sudo docker network create --scope=swarm --attachable -d overlay --opt encrypted coinswarmnet
```

```
osboxes@vps0:~$ sudo docker node ls
ID                                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
m8we3e4b19ua0uia4u75nurpe *     vps0       Ready    Active           Leader             27.5.1
f72xul4iuhha4x9x3edpv3og1       vps1       Ready    Active           Reachable          27.5.1
osboxes@vps0:~$ sudo docker network create --scope=swarm --attachable -d overlay --opt encrypted coinswarmnet
x555sf16ytk3laq92e8h8pyva
```

(*) Ta chọn tạo network trong cụm docker là có mã hóa (secure)

c. Liệt kê danh sách các network

```
sudo docker network ls
```

```
osboxes@vps0:~$ sudo docker network ls
NETWORK ID    NAME                DRIVER    SCOPE
f4e8313ca3e7  bridge             bridge    local
x555sf16ytk3  coinswarmnet       overlay   swarm
9dbbf91d8d99  docker_gwbridge    bridge    local
ad640ce5c8d0  host               host      local
fyzgvgzvdxl3w  ingress            overlay   swarm
1fa69d979c44  none              null      local
```

Câu hỏi: network có tên ingress là gì? Gợi ý: The ingress network is created automatically when you initialize or join a swarm. Most users do not need to customize its configuration, but Docker allows you to do so.

10. Tiến hành deploy hệ thống lên docker swarm: theo thứ tự redis, rng, hasher, worker, webui tương ứng với port và gắn vào coinswarmnet

```
sudo docker service create --name redis --network coinswarmnet redis
```

```
sudo docker service create --name rng -p 8001:80 --network coinswarmnet ismartio/rng:coinswarm
```

```
sudo docker service create --name hasher -p 8002:80 --network coinswarmnet ismartio/hasher:coinswarm
```

```
sudo docker service create --name worker --network coinswarmnet ismartio/worker:coinswarm
```

```
sudo docker service create --name webui -p 8000:80 --network coinswarmnet ismartio/webui:coinswarm
```

Nên tạo theo layer thấp nhất đến cao nhất (redis là cái tương tác cuối nên tạo trước)

```
verify: Service cuyx1xot28hsn1o9despint7w converged
osboxes@vps0:~$ sudo docker service create --name webui -p 8000:80 --network coinswarmnet ismartio/webui:coinswarm
p47ruqlianv33hguhmo1wb506
overall progress: 1 out of 1 tasks
1/1: running [=====]
verify: Service p47ruqlianv33hguhmo1wb506 converged
osboxes@vps0:~$
```

(*) Có thể sử dụng docker stack deploy để tạo các service lên docker swarm kết hợp với docker-compose

(*) Câu hỏi:

Tại sao ta cần khởi động các service theo thứ tự

Khi không khai báo tham số port cho service redis và worker thì mặc định sẽ là bao nhiêu?

Hạn chế của docker-compose khi deploy nhiều dịch vụ

11. Liệt kê các dịch vụ trên swarm

```
sudo docker service ls
```

```
osboxes@vps0:~$ sudo docker service ls

```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
zv62kpcjfyai	hasher	replicated	1/1	ismartio/hasher:coinswarm	*:8002->80/tcp
s9yh56vtyxpx	redis	replicated	1/1	redis:latest	
phs3tx3z8ozh	rng	replicated	1/1	ismartio/rng:coinswarm	*:8001->80/tcp
z3gmginvojyb	webui	replicated	1/1	ismartio/webui:coinswarm	*:8000->80/tcp
cuyx1xot28hs	worker	replicated	1/1	ismartio/worker:coinswarm	

```
osboxes@vps0:~$
```

Replicas (số nhân bản)

12. Xem dịch vụ rng đang chạy trên các node nào

`sudo docker service ps rng`

```
osboxes@vps0:~$ sudo docker service ps rng
ID            NAME      IMAGE              NODE    DESIRED STATE  CURRENT STATE      ERROR      PORTS
rb1jhsc70143  rng.1     ismartio/rng:coinswarm  vps1    Running        Running 24 minutes ago
```

Rng run trên NODE vps1 (tùy theo máy)

Nếu muốn xóa một dịch vụ ta dùng lệnh (ví dụ xóa rng) `sudo docker service rm rng`

(*) Lưu ý: Nếu có các dịch vụ chạy trên node có vai trò là manager (ví dụ vps0) có khả năng xảy ra tình trạng tiêu hao nhiều tài nguyên của manager node thì ta có thể chuyển các dịch vụ chạy sang các node có vai trò là worker.

`sudo docker node update --availability drain vps0`

(**) Đây là giải pháp để hệ thống High Availability (HA)

Trong bài k cần xoá dịch vụ nào...

Tùy tình huống, app có thể chạy trên máy manager luôn.

Lệnh trên là app, service nào, khi deploy lên docker swarm nằm trên master thì nó chuyển xuống cho worker.

13. Tiến hành Auto Scaling & Auto Load Balancing dịch vụ worker trên hệ thống

`sudo docker service scale worker=10`

```
osboxes@vps0:~$ sudo docker service scale worker=10
worker scaled to 10
overall progress: 10 out of 10 tasks
1/10: running [=====>]
2/10: running [=====>]
3/10: running [=====>]
4/10: running [=====>]
5/10: running [=====>]
6/10: running [=====>]
7/10: running [=====>]
8/10: running [=====>]
9/10: running [=====>]
10/10: running [=====>]
```

Giả sử cần tăng cường (scale) lên, quá quá nhiều jobs tập trung vào 1 dịch vụ thì nó sẽ tạo nhiều service để các yêu cầu không tập trung vào một mà phân tán ra.

14. Kiểm tra các dịch vụ

`sudo docker service ls`

```
osboxes@vps0:~$ sudo docker service ls
ID            NAME      MODE      REPLICAS  IMAGE              PORTS
zv62kpcjfyai  hasher    replicated 1/1        ismartio/hasher:coinswarm  *:8002->80/tcp
s9yh56vtyxpx  redis     replicated 1/1        redis:latest
phs3tx3z8ozh  rng       replicated 1/1        ismartio/rng:coinswarm     *:8001->80/tcp
z3gmgiimvojbv webui     replicated 1/1        ismartio/webui:coinswarm   *:8000->80/tcp
cuyx1xot28hs  worker    replicated 10/10     ismartio/worker:coinswarm
```

10/10 rep

5. Kiểm tra dịch vụ worker

`sudo docker service ps worker`

```
osboxes@vps0:~$ sudo docker service ps worker
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
ohwq96a88v0q	worker.1	ismartio/worker:coincswarm	vps1	Running	Running 11 minutes ago		
013tjma0bj3s	worker.1	ismartio/worker:coincswarm	vps0	Shutdown	Shutdown 11 minutes ago		
yrhrvp1y1wej	worker.2	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		
otgltd3i624q	worker.3	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		
lyhwvlqeg9pd	worker.4	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		
i31l5op7fenb	worker.5	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		
7twafdvdyj8f	worker.6	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		
rm73m1ea1s63	worker.7	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		
b4hxog13qvqu	worker.8	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		
3ctfcthel8wc	worker.9	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		
44u7ify9jdyn	worker.10	ismartio/worker:coincswarm	vps1	Running	Running 2 minutes ago		

```
osboxes@vps0:~$
```

Vps0 bị shutdown thì nó đẩy qua vps1 => HA

16. Sinh viên tự thao tác Auto Scaling & Load Balancing cho dịch vụ rng là 5 và hasher là 5

17. Sinh viên tự thao tác High Availability (HA) cho các máy vps1 và vps 2 đang nắm giữ vai trò là manager

18. Bật trình duyệt để xem kết quả

Câu hỏi:

- Sinh viên tiến hành kiểm tra xem dịch vụ webui đang hoạt động trên các node nào?
- Từ trình duyệt ở máy host (windows) hãy truy cập vào webui thông qua lần lượt các địa chỉ IP của các node sau:
 - o `http://<địa chỉ IP của các máy với vai trò là manager>:8000`
 - o `http://<địa chỉ IP của các máy với vai trò là worker đang có chạy dịch vụ webui>:8000`
 - o `http://<địa chỉ IP của các máy với vai trò là worker đang không chạy dịch vụ webui>:8000`

19. Hãy tạo thêm 5 máy ảo vps5, vps6, vps7, vps8, vps9 có cấu hình tương đương với các máy có vai trò worker, với địa chỉ IP tương ứng (192.168.56.106, 192.168.56.107, 192.168.56.108, 192.168.56.109, 192.168.56.110 hoặc tương đương). Thực hiện các thao tác tiến hành auto scaling, load balancing, high availability (HA)

- Trên hệ thống có 15 worker services, 10 rng services, 10 hasher services, 5 webui services và 2 redis services
- Thiết lập HA cho các máy có vai trò manager.

20. Thao tác tại các máy có vai trò là manager, hãy thực hiện:

- Tiến hành đo CPU/Memory usage và I/O usage
- Tiến hành đo latency của response time các dịch vụ rng và hasher

⇒ Từ máy manager, đo lường, sau đó đi qua từng máy để test. (20)

Phần 6: Hướng dẫn cài đặt ELK Stack trên Docker Swarm

- **Bước 1:** Bạn cần một **Docker Swarm Cluster** với nhiều node (ít nhất 1 manager và 1 worker).

Kiểm tra swarm bằng lệnh: *docker node ls*

```
osboxes@master01:~$ sudo docker node ls
[sudo] password for osboxes:
ID                HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS  ENGINE  VERSION
yqr7nc9g3wndo46amkxxuljyy * master01         Ready    Drain           Reachable        28.0.1
vmx4dvq9zj9zjlpdavbcb45v7 master02         Ready    Active          Reachable        28.0.1
2gx129vkm9ju366hmqlue70g5 master03         Ready    Active          Leader           28.0.1
ifjz7jhvbu5j5bjrox5mram3l vps01           Ready    Active          Reachable        28.0.1
td25pl4vzs5tn2co9u9od4ht4 vps02           Ready    Active          Reachable        28.0.1
pu922hoj0i6fvx1sc4zr50g9h vps03           Ready    Active          Reachable        28.0.1
k7olt0f7i62txgdumqbcht7g1 vps04           Ready    Active          Reachable        28.0.1
fwm77ef27yu9d52za3ji3xy6c vps05           Ready    Active          Reachable        28.0.1
69w8u56tcfls8ymu1snqdklej vps06           Ready    Active          Reachable        28.0.1
sj6lumi7wi307n4wk6u8phwdg vps07           Ready    Active          Reachable        28.0.1
osboxes@master01:~$
```

Nếu chưa có swarm, vui lòng tham khảo phần 4.

- **Bước 2:** Tạo Docker Network

Tạo một overlay network để các service ELK có thể giao tiếp với nhau.

[Sử dụng elk-app nó tự tạo network, bước này bỏ qua] *docker network create --driver overlay elk_network*

```
osboxes@master01:~$ sudo docker network create --driver overlay elk_network
uynsp7jn8r0b6bhoypuza678o
osboxes@master01:~$
```

- **Bước 3:** Tải file elk-app, Giải nén file ra sau đó đẩy lên master01

Mở cmd tại root (chứa elk-app, không mở trong elk-app):

Gõ: *scp -r elk-app osboxes@192.168.19.10:/home/osboxes*

- **Bước 4:** Triển khai ELK trên Docker Swarm

Đăng nhập master01

Chuyển đến folder vừa đẩy lên (/home/osboxes/elk-app), gõ

docker config create logstash_config logstash.conf

Sau đó deploy bằng lệnh *docker stack deploy -c docker-compose.yml elk*

Elk là tên stack, kệ nó, chả ảnh hưởng gì đâu.

```
osboxes@master01:~/elk-app$ sudo docker stack deploy -c docker-compose.yml elk
yaml: line 4: found character that cannot start any token
osboxes@master01:~/elk-app$ sudo docker stack deploy -c docker-compose.yml elk
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network elk_elk_network
Creating service elk_kibana
Creating service elk_elasticsearch
Creating service elk_logstash
osboxes@master01:~/elk-app$
```

Nếu gặp lỗi khi chạy, vui lòng kiểm tra định dạng thực tế, đầu dòng đúng format docker compose

- Bước 5: Kiểm tra Trạng Thái

Xem danh sách các service: *docker service ls*

```
osboxes@master01:~/elk-app$ sudo docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
vsce45vmf0d6	cadvisor	global	9/9	gcr.io/cadvisor/cadvisor:latest	
askih2pygnd2	elk_elasticsearch	replicated	0/1	docker.elastic.co/elasticsearch/elasticsearch:8.5.1	*:9200-
>9200/tcp					
ybnlyt41nvp3	elk_kibana	replicated	0/1	docker.elastic.co/kibana/kibana:8.5.1	*:5601-
>5601/tcp					
1byy20kw8mn4	elk_logstash	replicated	0/1	docker.elastic.co/logstash/logstash:8.5.1	*:5044-
>5044/tcp					
q1hf7t69rh36	hasher	replicated	10/10	kennex666/hasher:coinswarm	*:8002-
>80/tcp					
mxkj03wtn17q	redis	replicated	2/2	redis:latest	
kya1w992745o	rng	replicated	10/10	kennex666/rng:coinswarm	*:8001-
>80/tcp					
5gxh4qft4oql	webui	replicated	5/5	kennex666/webui:coinswarm	*:8000-
>80/tcp					
9ccmz61b895a	worker	replicated	15/15	kennex666/worker:coinswarm	

Xem logs của từng service:

docker service logs elk_elasticsearch

docker service logs elk_logstash

docker service logs elk_kibana

Câu log vào service:

[CŨ] *docker service update --log-driver=gelf --log-opt gelf-address=udp://192.168.19.10:12201 <service>*

VD câu vào rng: *docker service update --log-driver=gelf --log-opt gelf-address=udp://192.168.19.10:12201 rng*

[MỚI]

*sudo docker service update *

*--log-driver=gelf *


```
--log-opt gelf-address=udp://192.168.19.10:12201 \
```

```
--log-opt tag="{{.Name}}" \
```

```
rng
```

```
sudo docker service update \
```

```
--log-driver=gelf \
```

```
--log-opt gelf-address=udp://192.168.19.10:12201 \
```

```
--log-opt tag="{{.Name}}" \
```

```
worker
```

```
sudo docker service update \
```

```
--log-driver=gelf \
```

```
--log-opt gelf-address=udp://192.168.19.10:12201 \
```

```
--log-opt tag="{{.Name}}" \
```

```
webui
```

```
sudo docker service update \
```

```
--log-driver=gelf \
```

```
--log-opt gelf-address=udp://192.168.19.10:12201 \
```

```
--log-opt tag="{{.Name}}" \
```

```
hasher
```

Kiểm tra logstash (Không cần thiết, chờ khoảng 2-5 phút là được)

Gửi message tới logstash:

- `echo -e '\x1e\x00{"version":"1.1","host":"test","short_message":"Hello Logstash!","level":6}' | nc -u -w1 192.168.19.10 12201`
- `echo '{"version": "1.1", "host": "test", "short_message": "Hello Logstash"}' | nc -w 1 -u 192.168.19.10 12201`

Check log liên tục: `sudo docker service logs -f elk_logstash`

- **Bước 7:** Truy cập Kibana

Mở trình duyệt, vào: http://<IP_MANAGER>:5601

Bản 8.x tham khảo: <https://www.elastic.co/guide/en/kibana/8.17/data-views.html>

Các bước chính:

- Mở mục elastic -> Discover ->
- Pattern nhập: docker-*

<Câu hỏi: Làm sao để modify filter theo ý? (logstash.conf)>

Giám sát bảo trì <GPT>:

- Kiểm tra logs:

```
docker service logs elk_logstash --follow
```

- Kiểm tra trạng thái Elasticsearch:

```
curl -X GET "http://<IP_MANAGER>:9200/_cluster/health?pretty"
```

- Dừng stack:

```
docker stack rm elk
```

Lệnh remove để cài lại nếu lỗi:

```
osboxes@master01:~/elk-app$ sudo docker stack rm elk
```

```
osboxes@master01:~/elk-app$ sudo docker config rm logstash_config
```

```
osboxes@master01:~/elk-app$ sudo docker config create logstash_config logstash.conf
```

```
osboxes@master01:~/elk-app$ sudo docker stack deploy -c docker-compose.yml elk
```

```
sudo docker service logs -f elk_logstash
```

Lưu ý về log: **hasher, rng** sẽ gồm info cơ bản gồm ip, ngày, method http, trạng thái http, uri

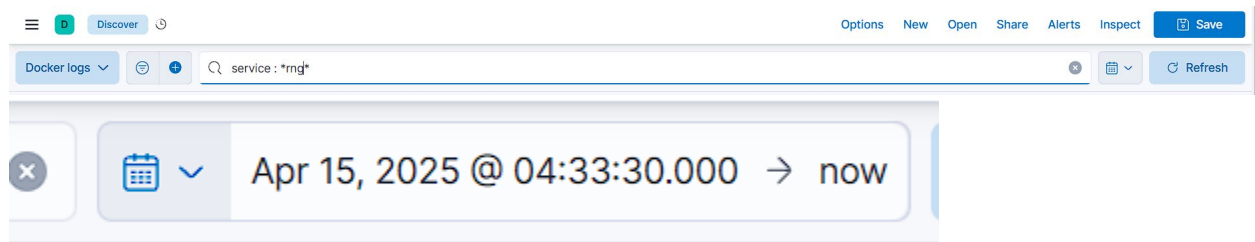
10.0.1.34 - - [14/Apr/2025 21:55:42] "GET /32 HTTP/1.1" 200 -

Webui có log liên quan đến redis khi bị lỗi, log khi khởi động. (message). Lưu ý, để thấy được log này sẽ rất khó vì không có log liên tục, dùng lệnh `sudo docker service scale webui=0` sau đó về 5 sẽ thấy log.

Worker sẽ có thông tin ``INFO: \_\_main\_\_:5 units of work done, updating hash counter``

Lưu ý: Tất cả đều parse về json hết, trừ webui do nó chỉ có message.

Cách để filter một service cụ thể: service: *<tên service>* sau đó nhấn refresh, nhớ kiểm tra thời gian từ khi nào?



Phần 7: Giám sát Docker Swarm

Đẩy các file trong folder monitor-swarm lên

scp -r monitor-swarm osboxes@192.168.19.10:/home/osboxes

Bước 1: Khởi tạo Mạng Overlay:

sudo docker network create --driver overlay monitoring-net

```
osboxes@master01:~/elk-app$ sudo docker network create --driver overlay monitoring-net  
sxc5dvhylgvvihxfrk0bftjve
```

Bước 2: Deploy node-exporter lên tất cả node

Chạy lệnh:

```
sudo docker service create \
  --name node-exporter \
  --mode global \
  --network monitoring-net \
  --mount type=bind,source=/proc,target=/host/proc,readonly \
  --mount type=bind,source=/sys,target=/host/sys,readonly \
  --mount type=bind,source=/,target=/rootfs,readonly \
  --publish mode=host,target=9100,published=9100 \
  prom/node-exporter \
  --path.procfs /host/proc \
  --path.sysfs /host/sys \
  --path.rootfs /rootfs
```

```
osboxes@master01:~/elk-app$ sudo docker service create \
  --name node-exporter \
  --mode global \
  --network monitoring-net \
  --mount type=bind,source=/proc,target=/host/proc,readonly \
  --mount type=bind,source=/sys,target=/host/sys,readonly \
  --mount type=bind,source=/,target=/rootfs,readonly \
  --publish mode=host,target=9100,published=9100 \
  prom/node-exporter \
  --path.procfs /host/proc \
  --path.sysfs /host/sys \
  --path.rootfs /rootfs
uz4c24gtyu3fh5dptxoxnfoc5
overall progress: 0 out of 9 tasks
ifjz7jhbvu5j: preparing [=====>]
fwm77ef27yu9: preparing [=====>]
2gx129vkm9ju: preparing [=====>]
k7olt0f7i62t: preparing [=====>]
vmx4dvq9zj9z: preparing [=====>]
69w8u56tcfls: preparing [=====>]
sj6lumi7wi30: preparing [=====>]
pu922hoj0i6f: preparing [=====>]
td25pl4vzs5t: preparing [=====>]
```

Sau đó kiểm tra: *sudo docker service ps node-exporter*

```
osboxes@master01:~/elk-app$ sudo docker service ps node-exporter
```

ID	NAME	ERROR	PORTS	IMAGE	NODE	DESIRED STATE	CURRENT
wwiydrgezpt4	node-exporter.2gx129vkm9ju366hmque70g5		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	master03	Running	Running
9 seconds ago							
c8391emq0xc4	node-exporter.69w8u56tcfls8ymulsnqdklej		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	vps06	Running	Running
20 seconds ago							
tbuwnwmhpj0o	node-exporter.fwm77ef27yu9d52za3ji3xy6c		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	vps05	Running	Running
23 seconds ago							
pl7piuqkld8c	node-exporter.ifjz7jhbvu5j5bjrox5mram3l		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	vps01	Running	Running
9 seconds ago							
wxk8326qfwpm	node-exporter.k7olt0f7i62txgdumqbcht7g1		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	vps04	Running	Running
24 seconds ago							
ggqt3js34bfd	node-exporter.pu922hoj0i6fvx1sc4zr50g9h		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	vps03	Running	Running
21 seconds ago							
da50177xpl8v	node-exporter.sj6lumi7wi307n4wk6u8phwdg		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	vps07	Running	Running
21 seconds ago							
h0z70f8x1kqm	node-exporter.td25pl4vzs5tn2co9u9od4ht4		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	vps02	Running	Running
12 seconds ago							
xbugcrmah90u	node-exporter.vmx4dvq9zj9zjlpdavbcb45v7		*:9100->9100/tcp,*:9100->9100/tcp	prom/node-exporter:latest	master02	Running	Running
18 seconds ago							

Bước 3: Tạo file cấu hình prometheus.yml

Tạo xong deploy config: *sudo docker config create prometheus.yml prometheus.yml*

```
osboxes@master01:~/monitor-swarm$ sudo docker config create prometheus.yml prometheus.yml
dl8738yzqdkut8g24qt90jqk1
```

Bước 4: Deploy docker compose của monitor

sudo docker stack deploy -c docker-compose.yml monitoring


```
osboxes@master01:~/monitor-swarm$ sudo docker stack deploy -c docker-compose.yml monitoring
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating service monitoring_prometheus
Creating service monitoring_influxdb
Creating service monitoring_grafana
```

Check lại: *sudo docker service ls*

Bước 5: Truy cập các giao diện setup lần đầu

<http://192.168.19.10:8086/>

Welcome — Initial User Setup — Complete

Setup Initial User

You will be able to create additional Users, Buckets and Organizations later

Username
osboxes

Password
.....

Confirm Password
.....

Initial Organization Name ⓘ
Nhom01-BigData

Initial Bucket Name ⓘ
Nhom01-BigData

CONTINUE

Welcome — Initial User Setup — Complete

You are ready to go!

Make sure to copy your operator API token now.
This token enables superuser privileges like creating users, orgs, etc. You won't be able to see it again!

JJ4tmzB3OnkR4yY1nhUj2NDJm5rfkP1lWZTe5P0rw8dzAEvMl6kD2-juhM6cQP2lInhughxG-aGtzRMiTo43eQ==

COPY TO CLIPBOARD

Your InfluxDB has 1 organization, 1 user, and 1 bucket.

Let's start collecting data!

QUICK START
Timing is everything!

ADVANCED
Whoa looks like you're an expert!

CONFIGURE LATER
I've got this...

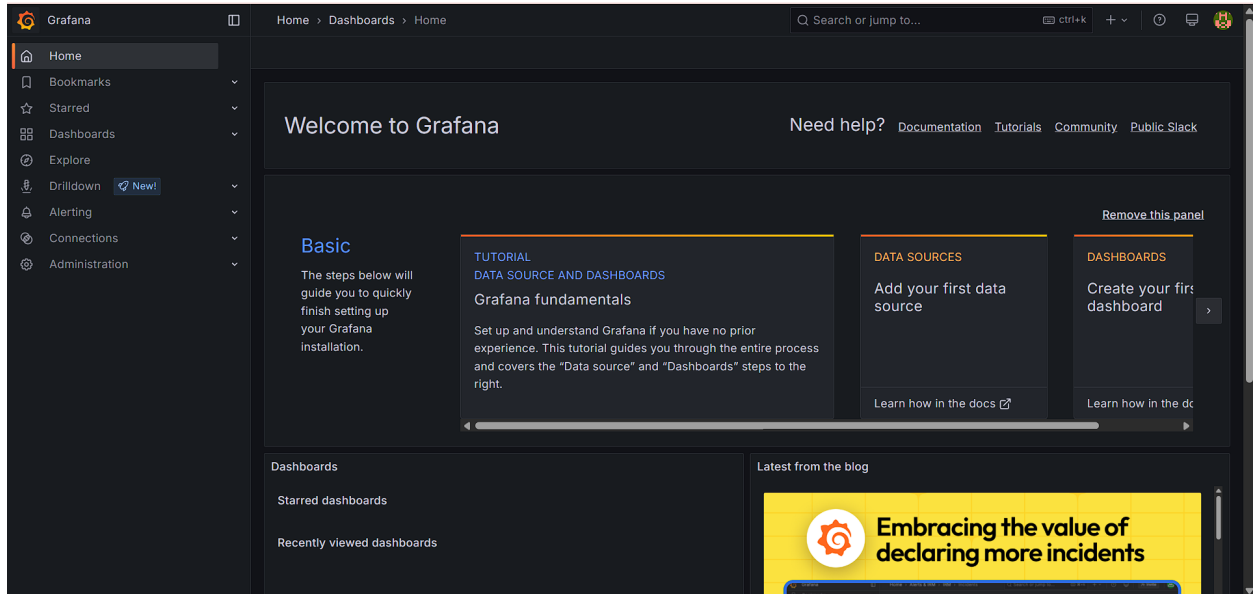
Powered by InfluxData

Token của Bảo: JJ4tmzB3OnkR4yY1nhUj2NDJm5rfkP1lWZTe5P0rw8dzAEvMl6kD2-juhM6cQP2lInhughxG-aGtzRMiTo43eQ==

<http://192.168.19.10:3000>

admin/admin

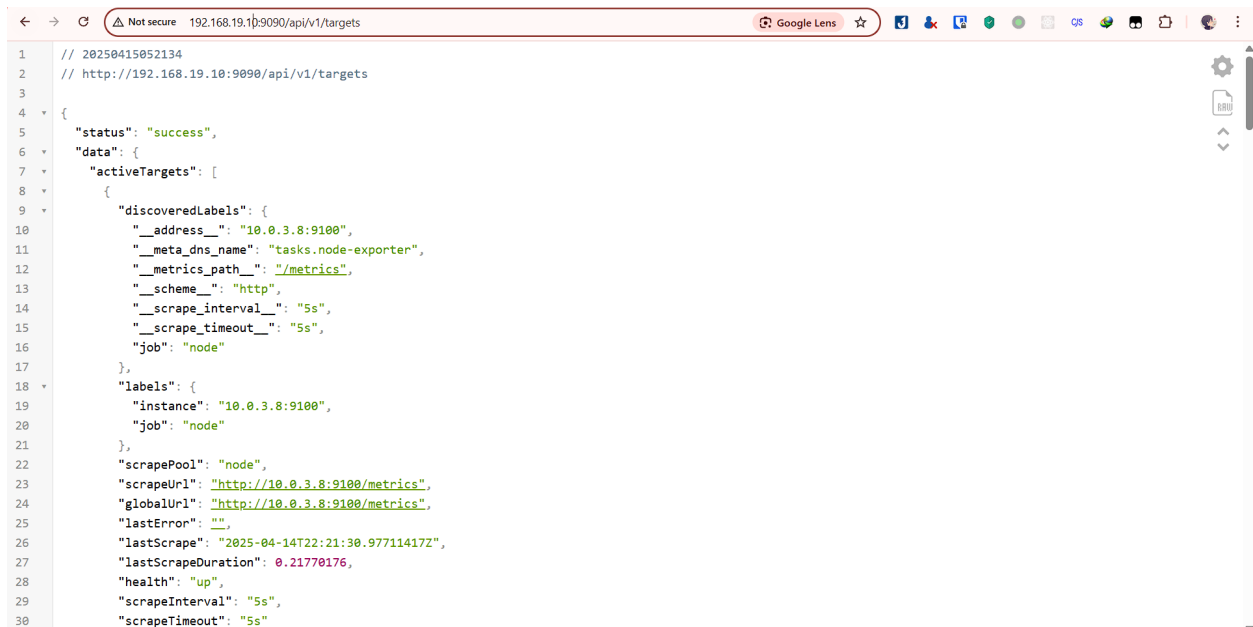
Đổi pass: osboxes.org



Bước 6: Check:

`curl http://localhost:9100/metrics` # kiểm tra node-exporter (Chỉ cần check cái dưới, trạng thái up hết là ok, kệ nó chạy đc metrics hay không)

`curl http://localhost:9090/api/v1/targets` # kiểm tra Prometheus đang scrape



Hướng dẫn giám sát:

Thứ nhất: <http://192.168.19.11:9100/metrics> (master02)

```
< > ↻ ⚠ Not secure 192.168.19.11:9100/metrics ☆
# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 2.1801e-05
go_gc_duration_seconds{quantile="0.25"} 4.6577e-05
go_gc_duration_seconds{quantile="0.5"} 5.5063e-05
go_gc_duration_seconds{quantile="0.75"} 6.5472e-05
go_gc_duration_seconds{quantile="1"} 0.00751612
go_gc_duration_seconds_sum 0.027344987
go_gc_duration_seconds_count 153
# HELP go_gc_gogc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the GOGC environment variable, and the runtime/debug.SetGCPPercent function. Sourced from /gc/gogc:percent
# TYPE go_gc_gogc_percent gauge
go_gc_gogc_percent 100
# HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environment variable, and the runtime/debug.SetMemoryLimit function. Sourced from /gc/gomemlimit:bytes
# TYPE go_gc_gomemlimit_bytes gauge
go_gc_gomemlimit_bytes 9.223372036854776e+18
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 8
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info(version="go1.23.7") 1
# HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.684136e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 2.58536752e+08
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.498904e+06
# HELP go_memstats_frees_total Total number of heap objects frees. Equals to /gc/heap/frees:objects + /gc/heap/tiny/allocs:objects.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 2.81662e+06
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata. Equals to /memory/classes/metadata/other:bytes.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 3.20464e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and currently in use, same as go_memstats_alloc_bytes. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 2.684136e+06
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used. Equals to /memory/classes/heap/released:bytes + /memory/classes/heap/free:bytes.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 3.72736e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use. Equals to /memory/classes/heap/objects:bytes + /memory/classes/heap/unused:bytes
# TYPE go_memstats_heap_inuse_bytes gauge
```

Truy cập <ip>:9100/metrics sẽ thấy info từng node.

Thứ 2. <http://192.168.19.10:9090/api/v1/targets>

Sẽ thấy info các node đang được giám sát

```
< > ↻ ⚠ Not secure 192.168.19.10:9090/api/v1/targets ☆ Google Lens
1 // 20250415052721
2 // http://192.168.19.10:9090/api/v1/targets
3
4 {
5   "status": "success",
6   "data": {
7     "activeTargets": [
8       {
9         "discoveredLabels": {
10           "__address__": "10.0.3.5:9100",
11           "__meta_dns_name": "tasks.node-exporter",
12           "__metrics_path__": "/metrics",
13           "__scheme__": "http",
14           "__scrape_interval__": "5s",
15           "__scrape_timeout__": "5s",
16           "job": "node"
17         },
18         "labels": {
19           "instance": "10.0.3.5:9100",
20           "job": "node"
21         },
22         "scrapePool": "node",
23         "scrapeUrl": "http://10.0.3.5:9100/metrics",
24         "globalUrl": "http://10.0.3.5:9100/metrics",
25         "lastError": "",
26         "lastScrape": "2025-04-14T22:27:17.755221488Z",
27         "lastScrapeDuration": 0.062325286,
28         "health": "up",
29         "scrapeInterval": "5s",
30         "scrapeTimeout": "5s"
```

Thứ 3: [http://192.168.19.10:9090/api/v1/query?query=node memory MemAvailable bytes](http://192.168.19.10:9090/api/v1/query?query=node%20memory%20MemAvailable%20bytes)

Xem ram đang trống

← → ↺ ⚠ Not secure 192.168.19.10:9090/api/v1/query?query=node_memory_MemAvailable_bytes ☆ 🔍 📄 📁 📂 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿

```
1 // 20250415052907
2 // http://192.168.19.10:9090/api/v1/query?query=node_memory_MemAvailable_bytes
3
4 {
5   "status": "success",
6   "data": {
7     "resultType": "vector",
8     "result": [
9       {
10        "metric": {
11          "__name__": "node_memory_MemAvailable_bytes",
12          "instance": "10.0.3.5:9100",
13          "job": "node"
14        },
15        "value": [
16          1744669746.132,
17          421961728
18        ]
19      },
20      {
21        "metric": {
22          "__name__": "node_memory_MemAvailable_bytes",
23          "instance": "10.0.3.4:9100",
24          "job": "node"
25        },
26        "value": [
27          1744669746.132,
28          468623360
29        ]
30      }
31    ]
32  }
33 }
```

Trình diễn pipeline thu thập và lưu metric

Pipeline gồm các bước:

- node-exporter thu thập từ host
- Prometheus scrape metric định kỳ qua cổng 9100
- Prometheus lưu trữ dữ liệu trong bộ nhớ time-series riêng
- Grafana truy vấn Prometheus để vẽ dashboard

Toàn bộ pipeline chạy tự động sau khi deploy stack trên Swarm.

✅ Mục tiêu:

Trình diễn **toàn bộ pipeline Prometheus** — từ thu thập → lưu trữ → truy vấn — **chỉ bằng CLI** (không cần mở web UI)

🔗 Cấu trúc pipeline CLI:

Giai đoạn	Công cụ CLI	Mô tả
1. Thu thập	curl node-exporter	Lấy metric từ host
2. Lưu trữ	curl Prometheus	Kiểm tra Prometheus đã scrape chưa
3. Truy vấn	curl PromQL API	Query dữ liệu

Các bước CLI cụ thể để demo pipeline:

1. Thu thập từ node-exporter

Chạy **trên đúng node** có container node-exporter:

```
curl http://localhost:9100/metrics | head -n 10
```

 Giải thích:

- Trả về các dòng metric kiểu:

```
# HELP node_memory_MemAvailable_bytes ...
```

```
node_memory_MemAvailable_bytes 3.273166848e+09
```

2. Kiểm tra Prometheus có scrape node-exporter không

```
curl -s http://localhost:9090/api/v1/targets | jq
```

 Trả về:

- Trạng thái up của node-exporter
 - Danh sách endpoint đang scrape
-

3. Truy vấn dữ liệu đã thu thập

Ví dụ: xem dung lượng RAM khả dụng trong 5 phút gần nhất:

```
curl -G http://localhost:9090/api/v1/query \  
--data-urlencode 'query=node_memory_MemAvailable_bytes'
```

Hoặc dùng range query để xem theo thời gian:

```
curl -G http://localhost:9090/api/v1/query_range \  
--data-urlencode 'query=node_cpu_seconds_total' \  
--data-urlencode 'start=2025-01-01T00:00:00Z' \  
--data-urlencode 'end=2025-01-01T00:05:00Z' \  
--data-urlencode 'step=10s'
```

```
--data-urlencode 'start=2025-04-14T15:00:00Z' \
```

```
--data-urlencode 'end=2025-04-14T15:10:00Z' \
```

```
--data-urlencode 'step=30s'
```

Khi thuyết trình:

Em sẽ trình diễn quy trình pipeline giám sát Prometheus sử dụng thuần CLI:

- Dùng curl truy vấn trực tiếp vào node-exporter để xem raw metric
 - Dùng Prometheus API (/api/v1/targets) để xác nhận metric được thu thập
 - Dùng PromQL API để truy vấn dữ liệu đã được lưu, ví dụ như CPU hoặc RAM → Như vậy có thể triển khai & kiểm tra pipeline giám sát hoàn toàn qua dòng lệnh
-

Giới thiệu InfluxDB

InfluxDB là hệ quản trị cơ sở dữ liệu dạng time-series

Chuyên dùng để lưu trữ dữ liệu theo thời gian như:

- CPU usage theo thời gian
- Nhiệt độ, sensor, logs IoT

 Triển khai bằng Docker Swarm tại cổng 8086, có GUI quản lý trực quan.

Đồng bộ hóa dữ liệu từ Prometheus qua InfluxDB

Sử dụng remote_write trong cấu hình prometheus.yml:

yaml

remote_write:

```
- url: "http://influxdb:8086/api/v2/write?org=...&bucket=...&precision=s"
```

```
  bearer_token: "..."
```

 Cần setup token và bucket trong InfluxDB v2.

Dữ liệu sẽ được Prometheus gửi thẳng qua Influx.

1. Tạo token & bucket bên InfluxDB

1. Truy cập: <http://localhost:8086>

2. Đăng nhập và tạo:

- **Organization:** demo-org
 - **Bucket:** prometheus
 - **Token:** ABCDEF... (ghi lại để xài)
-

2. Cập nhật prometheus.yml

global:

scrape_interval: 5s

remote_write:

- url: "http://influxdb:8086/api/v2/write?org=demo-org&bucket=prometheus&precision=s"

bearer_token: "<TOKEN_CỦA_EM>"

scrape_configs:

- job_name: 'node'

dns_sd_configs:

- names: ['tasks.node-exporter']

type: 'A'

port: 9100

👉 Nhớ thay <TOKEN_CỦA_EM> bằng token Influx thật.

3. Tạo lại config trên Swarm

docker config rm prometheus.yml

docker config create prometheus.yml prometheus.yml

docker service update --force monitoring_prometheus

4. Kiểm tra InfluxDB nhận dữ liệu chưa

Dùng CLI hoặc GUI:

```
influx query 'from(bucket:"prometheus") |> range(start: -10m)'
```

🔥 Mô tả gọn để viết báo cáo hoặc nói miệng:

Để đồng bộ dữ liệu từ Prometheus sang InfluxDB, em sử dụng cơ chế `remote_write`.

Trong file `prometheus.yml`, em cấu hình Prometheus gửi dữ liệu định kỳ đến API `/api/v2/write` của InfluxDB, kèm theo `org`, `bucket`, và `token` xác thực.

Khi deploy lại, Prometheus sẽ tự push dữ liệu sang InfluxDB — từ đó em có thể truy vấn và trực quan hóa bằng Grafana.

Truy vấn dữ liệu từ InfluxDB qua CLI

Dùng CLI hoặc GUI để truy vấn:

```
influx query 'from(bucket: "metrics") |> range(start: -1h)'
```

📊 Có thể xem dữ liệu CPU, RAM, network theo node hoặc container.

📖 Cách truy vấn bằng CLI

◆ 1. SSH hoặc vào container influxdb:

```
docker exec -it monitoring_influxdb.1.xxxx /bin/bash
```

◆ 2. Chạy CLI truy vấn:

```
influx query \
```

```
'from(bucket:"prometheus")
```

```
|> range(start: -5m)
```

```
|> filter(fn: (r) => r._measurement == "node_memory_MemAvailable_bytes")'
```

🔥 Có thể thay bằng bất kỳ metric nào như:

- `node_cpu_seconds_total`
- `node_disk_io_time_seconds_total`
- `node_network_receive_bytes_total`

📊 Truy vấn bằng InfluxDB GUI:

1. Mở trình duyệt: <http://localhost:8086>
2. Vào **Data Explorer**

3. Chọn:

- Bucket: prometheus
 - Measurement: Chọn metric như CPU, RAM...
 - Thời gian: Last 5 minutes
-

4. Xem kết quả dạng bảng hoặc biểu đồ

Giải thích để báo cáo / trình bày miệng:

Sau khi Prometheus đẩy metric vào InfluxDB qua API `remote_write`, em có thể truy vấn dữ liệu này bằng câu lệnh Flux trong CLI hoặc trực tiếp trên giao diện Explorer của Influx.

Ví dụ truy vấn RAM khả dụng trong 5 phút gần nhất:

flux

CopyEdit

```
from(bucket:"prometheus")
```

```
|> range(start: -5m)
```

```
|> filter(fn: (r) => r._measurement == "node_memory_MemAvailable_bytes")
```

Đây là bảng chứng dữ liệu đã lưu thành công và truy xuất được từ InfluxDB chạy trên Swarm.

Tóm tắt:

Cách Ưu điểm

CLI	Đáp ứng yêu cầu "thao tác bằng lệnh"
-----	--------------------------------------

GUI	Trình bày đẹp khi chụp hình demo
-----	----------------------------------

Giới thiệu Grafana

Grafana là công cụ mạnh để trực quan hóa dữ liệu:

- Hỗ trợ nhiều nguồn: Prometheus, InfluxDB, MySQL...
- Cho phép vẽ biểu đồ, bảng, gauge, heatmap
- Có thể gửi cảnh báo nếu vượt ngưỡng

 Giao diện đẹp, thân thiện, dễ tùy biến dashboard.

Tạo dashboard phân tích hệ thống

Import dashboard có sẵn (ID: 1860 – Node Exporter Full)
Hoặc tự tạo dashboard gồm:

- CPU usage từng node
- RAM, Disk, Network I/O
- Tổng số process, container đang chạy

🌸 Dashboard giúp phân tích hiệu suất toàn hệ thống Docker Swarm.

Mục tiêu:

Sử dụng Grafana + Prometheus → vẽ dashboard để phân tích hiệu suất như:

- CPU, RAM, Disk, Network
 - Trạng thái các node
 - Tình trạng container / process / uptime
-

🔧 Step-by-step tạo Dashboard giám sát Docker Swarm:

◆ 1. Truy cập Grafana

Link: <http://localhost:3000>

Đăng nhập mặc định: admin / admin

◆ 2. Thêm Data Source Prometheus

1. Vào  **Configuration** → **Data Sources**
 2. Chọn **Prometheus**
 3. Nhập URL: `http://prometheus:9090`
 4. Click **Save & Test**
-

◆ 3. Import dashboard mẫu cực đẹp (Node Exporter Full)

1. Vào + → **Import**
 2. Dán ID sau: 1860
 3. Click **Load**
 4. Chọn Prometheus data source đã tạo
 5. Nhấn **Import**
-

🎯 Kết quả: Dashboard giám sát đầy đủ:

- Tổng CPU usage
 - RAM các node
 - Load trung bình
 - Network I/O
 - Disk I/O
 - Container & process count
 - Trạng thái node Swarm
-

Mô tả gọn để ghi trong báo cáo hoặc nói miệng:

Em đã sử dụng Grafana để tạo dashboard phân tích hiệu suất hệ thống Swarm bằng dữ liệu từ Prometheus.

Dashboard gồm nhiều biểu đồ hiển thị các thông số quan trọng như:

- Sử dụng CPU và RAM theo từng node
 - Tốc độ đọc/ghi đĩa
 - Lưu lượng mạng gửi/nhận
 - Tổng số container và tiến trình đang chạy
-

Dữ liệu được cập nhật realtime mỗi 5 giây, hỗ trợ tốt cho việc phát hiện sớm vấn đề hiệu suất trong hệ thống.

Tips bonus:

- Muốn tạo dashboard riêng: Vào + → **Dashboard** → **Add Panel**
 - Dùng query PromQL ví dụ:
-

```
sum(rate(node_cpu_seconds_total{mode!="idle"}[1m])) by (instance)
```
